Market Regimes for Distribution Shift in a Cryptocurrency Two-Stage Learning Architecture

Vincent Joulain

National Institute of Informatics, Tokyo, Japan
vincent.joulain@protonmail.com

Abstract. Financial models deployed in cryptocurrency markets face persistent non-stationarity and distribution shift. This work investigates whether market regimes can improve a two-stage forecasting architecture in which a secondary classifier filters predictions from the Kronos foundation model. Statistical Jump Models are used to identify persistent market states at daily and 4-hour scales. Adapted 4-hour regime features produced higher median mean trade returns than their corresponding baselines in 23/25 validation configurations and 16/25 test configurations. More consistently, validation-selected regime gating improved both precision and mean accepted-trade return in 19/20 evaluated settings. Distribution analysis showed that regimes strongly partition the feature space, while regime-conditioned feature and seed selection did not transfer reliably out of sample. Overall, regimes are most effective as a diagnostic and trade-filtering layer rather than a standalone solution to distribution shift.

Table of Contents

Market Regimes for Distribution Shift in a Cryptocurrency Two-Stage Learning Architecture . . . 1
Vincent Joulain
1 National Institute of Informatics . . . 3
1.1 Prendinger's laboratory . . . 3
Professor Helmut Prendinger . . . 3
The lab . . . 3
Internship objective . . . 3
2 Introduction . . . 4
2.1 Financial time series . . . 4
2.2 Two-stage learning architecture . . . 4
2.3 Out-Of-Distribution problem . . . 5
Distribution shifts . . . 6
How to address data shifts . . . 6
3 Exploratory work . . . 8
3.1 Shift reduction methods . . . 8
Specialized streaming models . . . 8
Invariant features . . . 9
Online drift detection and response . . . 9
Adaptive Conformal Inference . . . 10
Per regime classification . . . 10
4 Regime classification . . . 11
4.1 Statistical Jump Models . . . 12
Clustering loss and jump penalty . . . 12
Coordinate descent . . . 12
Sparse feature selection . . . 13
BIC selection . . . 14
FTIC selection . . . 16
Jump Models features and fitting . . . 16
5 Methodology . . . 17
5.1 Feature sets . . . 17
5.2 Complexity reduction . . . 18
5.3 Walk-forward cross-validation . . . 18
6 Experiments . . . 19
6.1 Jump Models regime assignment . . . 19
6.2 Regime features . . . 21
Distribution distances . . . 23
6.3 Regime gating . . . 23
7 Complementary experiments . . . 25
7.1 M2 feature analysis . . . 25
7.2 Seed selection . . . 28
8 Future steps . . . 29
8.1 On regime as a feature . . . 29
8.2 On regime gating . . . 29
9 Conclusion . . . 30
A Annexes . . . 31
A.1 Distribution distances across regimes . . . 31
A.2 Results of M1 and M2 across regimes . . . 32
A.3 Regime gating results . . . 34
A.4 Results on the fixed *train*, *val*, *test* splits . . . 37
A.5 Results on the walk-forward cross-validation splits . . . 40

1 National Institute of Informatics

The National Institute of Informatics (NII) is Japan's premier inter-university research institute dedicated to advanced studies in informatics and computer science. Operating under the Research Organization of Information and Systems (ROIS), NII plays a key academic role through its partnership with the Graduate University for Advanced Studies (SOKENDAI). As one of SOKENDAI's constituent institutes, NII supervises and trains doctoral students in informatics, offering a unique research-driven curriculum closely tied to ongoing projects at the institute. This integration allows students to work directly with leading researchers, participate in international collaborations, and gain hands-on experience with state-of-the-art computational facilities. Through this dual role of conducting advanced research while educating the next generation of scholars, NII and SOKENDAI jointly advance Japan's leadership in informatics and data science on the global stage.

1.1 Prendinger's laboratory

Professor Helmut Prendinger Helmut Prendinger received his M.S. and Ph.D. in Logic and Artificial Intelligence from the University of Salzburg, Austria. After positions as a research associate and JSPS postdoctoral at the University of Tokyo, and as a Junior Specialist at the University of California, Irvine, he is now a Full Professor at NII, Tokyo. His research spans intelligent drone systems and machine learning, with a focus on deep learning and foundation models for stock and cryptocurrency price prediction. He has published over 260 refereed papers and holds an H-index of 50. Prof. Prendinger was my internship supervisor during my stay at NII.

The lab Over the recent years, Professor Prendinger's laboratory has mainly been working on financial time series forecasting over cryptocurrency markets. During the time I was there, the laboratory was composed of NII Ph.D. student Pablo Molla, NII Ph.D. student Albi Isufaj and Ph.D. Intern Till Konrad. We worked together on this project, focusing on different areas, throughout my internship. Weekly meetings were held to discuss project progress and recent achievements.

Internship objective The objective of this internship was to address distribution shifts in financial time-series data (see Section 2.3). This is a well-known challenge in time-series modeling, and particularly in finance, where the signal-to-noise ratio is low and exploitable patterns are inherently difficult to identify. Given this constraint, the objective was to investigate state-of-the-art methods for mitigating distribution shifts, identify the most promising approach, and integrate it into the existing model to improve downstream forecasting performance.

2 Introduction

2.1 Financial time series

A financial time series is a sequence of financial values (i.e. asset prices, returns, or economic indicators) ordered by time and typically taken at consistent intervals. In this work, the initial time series data consisted of Open, High, Low, Close, Volume and Amount (OHLCVA) extracted from the Binance API over 20 assets and 8 time granularities (daily, 12h, 8h, 6h, 4h, 2h, 1h, 30 min) and individually downloaded external features. To train tabular models and extract meaningful information from temporal data, the time series was transformed into engineered features as presented in Fig. 1. The size of the lookback window depends on the considered time frame, ranging from 40 bars at the daily granularity to 90 bars at the 30-minute granularity (approximately two days). The engineered features include technically derived economic indicators such as return moments, realized and range-based volatility, momentum and oscillator readings, and volume-flow indicators, together with external features such as the Deribit Volatility Index and crypto risk premium, the Fear & Greed and news-sentiment indices, and ten macro-financial series entered as causal z-scores: uncertainty (EPU, EMU), volatility (VIX, OVX), rates (10-year yield, 3-month bill, term spread) and inflation expectations (breakeven, 5y-5y forward).

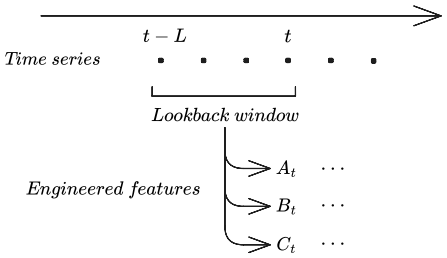


Fig. 1. Engineered Features

2.2 Two-stage learning architecture

The two-stage learning architecture, originally introduced in the context of financial machine learning in *Advances in Financial Machine Learning* by de Prado (2018) [8] as the *Meta-Labeling framework*, provides a pipeline for enhancing forecasters by filtering unreliable predictions. It is composed of a primary model (M1) maximising opportunity (high recall) under a secondary model (M2) filtering unreliable forecasts (high precision). The primary model is a foundational time series forecaster, *Kronos-base* introduced in *Kronos: A Foundation Model for the Language of Financial Markets* by Shi et al. (2026) [23]. The model takes as input a sequence of historical K-line data and autoregressively generates candles until the forecast horizon is reached, hence not only producing a price movement but also a price forecast. In this work, the forecast horizon is set to 7 candles, corresponding to a week in daily granularity. The primary model forecasts are then separated in two isolated sets: *Up* or *Down* according to the predicted return 7 bars ahead. Forecasts whose absolute predicted return does

not exceed the transaction fees threshold for entering and exiting the market are discarded because the model does not forecast sufficient edge to justify entering the trade. The decision to distinguish market directions was motivated by the premise that bull and bear regimes are driven by substantially different behaviors.

Up and down price movement predictions respectively compose the *Up* and *Down* datasets. Because the M1 predictions are thresholded, the meta-labeled samples (whether M1 prediction was directionally right 1 or wrong 0) constitute a discontinuous subset of the original time series. The discontinuity does not prevent tabular fitting because each observation contains lag-derived features; temporal leakage and dependence are instead controlled through chronological splitting and horizon-based purging.

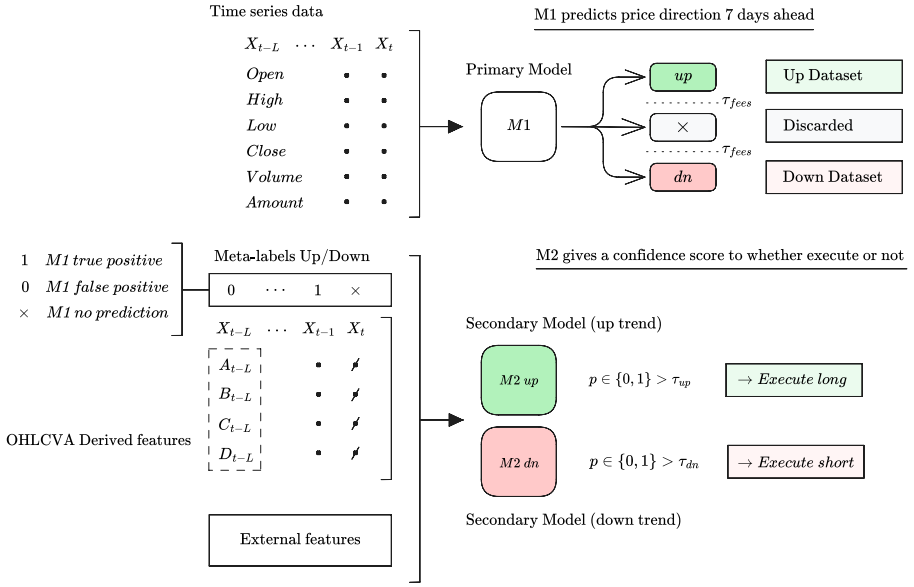


Fig. 2. Two-stage learning pipeline

2.3 Out-Of-Distribution problem

Machine Learning models often make the assumption of independent and identically distributed (IID) data, which posits that training and testing data are drawn from the same distribution. However, time series are characterized by temporal dependency and non-stationarity, which violate the IID assumption. In consequence, pre-trained models (on historical data) may experience significant performance degradation when deployed on dynamic and evolving data.

Distribution shifts Distribution shifts are changes in the data from the training stage and the testing stage. They can either be sudden, gradual or cyclic. There are three main types of distribution shifts :

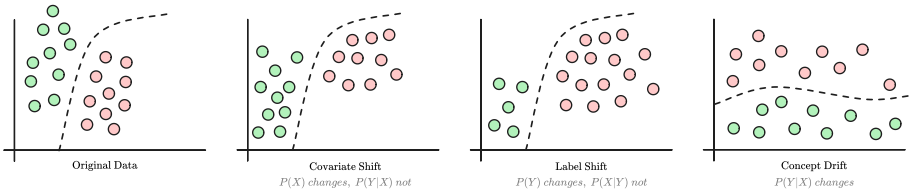


Fig. 3. Distribution Shifts

– Covariate shift is a displacement of the distribution of input data. A theoretical perfect model that has learned the exact relationship $P(Y|X)$ should not suffer any performance drop from this type of shift. However, in real-world settings and finite data, models approximate this relationship and could be prone to an important decrease in performance upon this type of shift.

$$P^{train}(X) \neq P^{test}(X), \qquad P^{train}(Y|X) = P^{test}(Y|X)$$

– Label shift is a change in the distribution of assigned classes (target variables). A model may be at risk of miscalibration and therefore degraded performance if not adjusted accordingly.

$$P^{train}(Y) \neq P^{test}(Y), \qquad P^{train}(X|Y) = P^{test}(X|Y)$$

– Concept drift is a modification of the underlying relationship between X (inputs) and Y (outputs) over time. Facing such a drift, the model needs a way to update its understanding of the rules of the problem if it wants to maintain accurate predictions.

$$P^{train}(Y|X) \neq P^{test}(Y|X)$$

How to address data shifts Distribution shifts are a known and hard problem in time series data and particularly in finance [21] [27]. A recent survey *Out-of-Distribution Generalization in Time Series: A Survey* by Wu et al. (2026) [28] provides a comprehensive review of methods for mitigating distribution shifts. These methods are divided by their authors into four categories: Decoupling-based Methods, Invariant-based Methods, Adaptive Mechanism-based Methods and Large Time Series Models.

Decoupling-based methods. These approaches aim to separate features into stable (task-relevant) and unstable (task-irrelevant) components, thereby improving model robustness and generalization.

The input is split as follows: $x = x_{\text{stable}} + x_{\text{unstable}}$ The model aims to learn: $P(y \mid x_{\text{stable}})$

Available strategies include multi-structured decomposition, which separates the time series into independent and interpretable components such as trend, seasonality, and noise, and causality-inspired decoupling, which aims to distinguish causal features representing the true underlying signal from spurious, environment-dependent correlations.

Invariant-based methods. These approaches aim to learn feature representations that remain stable (invariant) across environments or different data distributions, so that a model can generalize to unseen domains.

A transformation $\phi(x)$ of the input is learned such as:

$$P(y \mid \phi(x)) \text{ is stable across environments}$$

Available strategies include Invariant Risk Minimization [1], which learns representations for which the same predictor remains optimal across different environments, thereby encouraging invariant input-to-output relationships, and domain-invariance models, which align feature distributions across domains to reduce environment-specific variations.

Adaptive mechanism-based methods. Adaptive Mechanism-based Methods are approaches that enable a model to adapt dynamically to changing data distributions in non-stationary environments, permitting real-time adaptations.

Available strategies include explicit adaptation mechanisms, where the model's parameters, outputs, or structure are updated in response to newly arriving data through a predefined adaptation policy, such as online learning, and implicit adaptation mechanisms, where adaptation is embedded directly in the model architecture and occurs without explicit update rules, as in adaptive architectures.

Large Time Series Model. LTSMs are large-scale models trained on massive and diverse time series datasets, with the objective of learning general and transferable representations of temporal patterns, thereby improving robustness to distribution shifts.

Available strategies include LLM-based methods, which adapt pre-trained language models to time-series tasks by transforming temporal observations into representations or tokens compatible with the LLM, and Time Series Foundation Models, which are trained directly on large-scale and diverse time-series datasets to learn general-purpose temporal representations that can transfer across downstream tasks.

3 Exploratory work

The primary forecaster, *Kronos-base*, is a foundation model belonging to the LTSM category. It is therefore assumed to demonstrate at least moderate robustness to distributional shifts. In contrast, the secondary model is a lightweight and task-specific classifier that is more susceptible to distribution shifts. Consequently, focusing the mitigation strategy on the secondary classifier was considered the more practical choice. Initially, multiple options were considered: specialized streaming models; feature derivation using an invariant-based method; online drift detection and correction; adaptive conformal thresholds; and finally per regime classification.

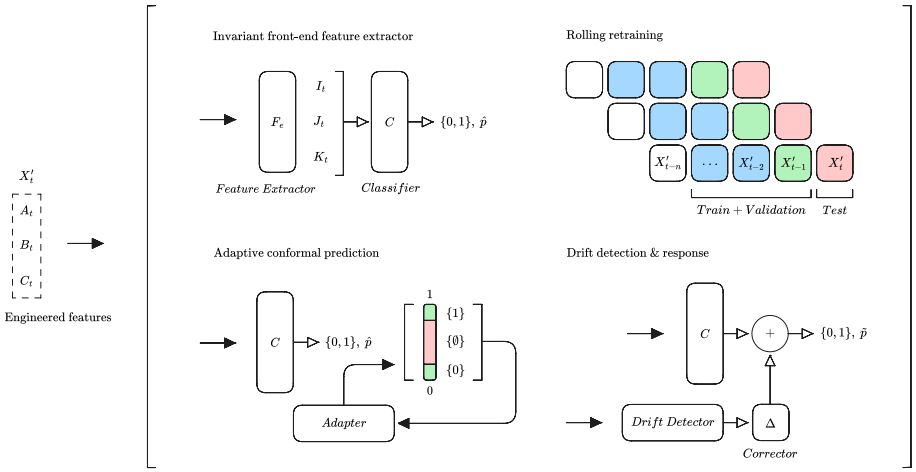


Fig. 4. Exploratory work, $\hat{p}$ and $\tilde{p}$ are respectively the model probability and corrected probability given to class 1.

3.1 Shift reduction methods

Specialized streaming models Specialized streaming models (*Adaptive Random Forests* [12], *Streaming Random Patches* [13] and *Streaming Gradient Boosted Trees* [14]) inherently handle data streams and are particularly efficient when concept drift is genuinely strong and persistent. However, failure on the task is not easily interpretable (no decomposition into layers to monitor where and why the models failed), and they may overreact to sudden and temporary market changes. Furthermore, exploratory work on rolling retraining displayed weak results (models trained on rolling windows of 2,000, 5,000, and 10,000 observations often underperformed the baseline model on most of the granularities on precision and mean trade return). Finally, the M2 model is subject

to change, and tying model selection to distribution handling would reduce the overall flexibility on the project.

Invariant features The underlying assumption was that the features driving the market, potentially providing an edge, were precisely those drifting features. In an extremely low signal-to-noise dataset, removing this drift risked eliminating valuable information, making this direction less relevant to pursue.

Online drift detection and response Following the Adaptive Mechanism-based Methods, online drift detection and response emerged from the study of several frameworks presented in the previously referenced survey. Most of these frameworks were originally designed to adapt the weights of deep learning forecasting models. However, their drift-detection component can be decoupled from the subsequent model correction mechanism. This enables the use of a model-agnostic wrapper on top of any classifier: drift can be detected through concept differences in the latent representation of samples (*Proceed* [29]), context-driven prediction errors (*SOLID* [5]), or conditional shifts among features (*FSD* [16]). The classifier's output probabilities can then be adjusted accordingly. The idea would have been the following:

$$\tilde{p} = \sigma(s + \omega \cdot \Delta)$$

$$with:$$

$$s = logit(\hat{p}) = log\left(\frac{\hat{p}}{1-\hat{p}}\right)$$

Where $\tilde{p}$, $\hat{p}$ are respectively the corrected and base model prediction probabilities, σ the sigmoid function, Δ the adapter correction and ω the gating strength from the detector confidence (whether the drift score is high/low and if there is a need to take it into consideration). Nonetheless, this method can only affect the model by modifying its probability and cannot tackle true concept drift. It effectively acts as a lightweight calibrator layer. Given this limitation, the implementation complexity appeared to outweigh the potential benefits, particularly when compared with adaptive conformal inference methods, despite its advantage of being potentially proactive rather than purely reactive and possibly limiting the feedback delay issue.

Adaptive Conformal Inference Adaptive Conformal Inference (ACI) is the action of converting a prediction into a prediction set based on a threshold representing the confidence of the model (larger if less confident, narrower if more). First, the conformity score is calculated for each candidate class:

$$S_t(0) = \hat{p}_t \qquad S_t(1) = 1 - \hat{p}_t$$

The prediction set is constructed as follows:

$$\hat{C}_t = \{\, y \in \{0, 1\} : S_t(y) \leq \tau_t \,\}$$

In standard ACI, the step size parameter controlling how fast the model reacts is fixed which makes adaptation to arbitrary shifts difficult. *DtACI* [11] is a version of ACI where the step size is learned online by combining several candidates, therefore allowing it to handle any type of shifts. Each expert i has its own adaptive parameter $\alpha_t^{(i)}$:

$$\alpha_{t+1}^{(i)} = \alpha_t^{(i)} + \gamma_i(\alpha - err_t^{(i)})$$

Where err_t indicates whether the last prediction set missed the true label, α is the target miscoverage level ($1 - coverage$) and γ is the parameter controlling how fast the model reacts (step size). The final output $\bar{\alpha}_t$ is calculated using the combination of probabilities derived from each expert's weights. These weights are updated using a pinball loss:

$$\bar{\alpha}_t = \sum_i p_t^{(i)} \alpha_t^{(i)}$$

Finally, the threshold is computed as:

$$\tau_t = Quantile(1 - \bar{\alpha}_t, D_t^y)$$

Where D_t^y denotes the reference distribution of recent conformity scores. Conformal inference is a relevant approach within this two-stage learning framework and could potentially be combined with regime classification. However, regime-based methods appeared more promising in this setting, as conformal prediction primarily acts as a reactive layer, adjusting decisions only after prediction errors have been observed. In a particularly noisy environment, where the error rate is close to 49.9%, a sufficiently large number of observations is required before distinguishing meaningful performance degradation from random fluctuations. This creates a difficult trade-off between reacting quickly to genuine distribution shifts and overreacting to noise, making the choice of an appropriate adaptation horizon particularly challenging.

Per regime classification The idea of addressing drift by modeling market regimes separately is supported by recent literature [20] [9]. However, there is no consensus on a standard methodology for defining regimes such as *uptrend*, *downtrend*, or *sideways*. Regime identification appears reasonably reliable for

coarse partitions, while stability generally decreases as the number of states increases, with two to five regimes often providing the most effective trade-off for capturing structural changes.

Regime classification can be approached in several ways. Simple heuristics may rely on market trends, volatility, or stress indicators. Markov-switching models provide a probabilistic framework in which a sequence of latent market states is inferred from observable data. Alternatively, change-point detection methods can identify abrupt shifts in the statistical properties of a time series, partitioning it into homogeneous segments that can subsequently be assigned to specific regimes, for instance using a trend-based score. Since Hidden Markov Models and the more recently introduced *Statistical Jump Models* represent prominent approaches for identifying regimes in financial time series [26], this direction was selected for further investigation.

4 Regime classification

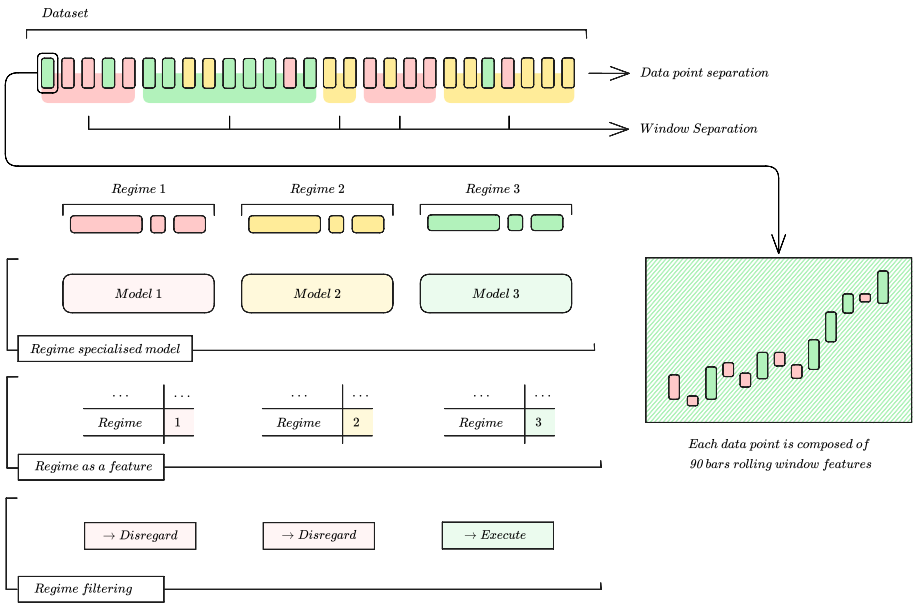


Fig. 5. Regime Adaptive Mechanisms

Financial market regimes such as bull market, bear market or range-bound market are distinct and persistent states of an economy or trading environment defined by specific patterns of volatility, asset returns, and macroeconomic factors (inflation, unemployment, interest rates, etc.). Regime classification consists

of assigning each time series data point a specific regime, usually this was done using Hidden Markov Models (HMM) to derive the hidden state of the market from observable variables. In recent work, statistical jump models have been used for this endeavor by clustering observations with a predefined jump penalty for switching regimes. Hyperparameter selection of such models could either be achieved through a statistical fit (Information Criterion) [6] [7] or a financial fit (given a naive or complex strategy) [24].

4.1 Statistical Jump Models

Statistical jump models are machine learning clustering tools used to break time series into distinct periods, with an explicit penalty for switching regimes. They initially appeared as a general mathematical framework in *Fitting Jump Models* by Bemporad et al. (2018) [4]. In this paper, Bemporad et al. showed that HMMs could be represented as a special case of the broader jump-model architecture. Two years later, these models were first applied to financial regime classification [19] by combining temporal-feature clustering with an explicit jump penalty designed to produce persistent and interpretable market regimes. Sparse jump models [18] subsequently enabled importance feature selection by assigning weights based on Between Clusters Sum of Squares (BCSS). More recently, jump models were extended to provide soft regime probabilities (Continuous Jump Models) [2]. In this study, the continuous variants were disregarded as the complexity of the probability grid grows exponentially with the number of clusters used, thereby limiting the practical number of clusters to three or four. Furthermore, the sparse jump model was selected because it allowed the model to assign weights to individual features.

Clustering loss and jump penalty At each time t with T the number of total observations, the model assigns the feature vector v_t to one of K states $s_t \in \{0, \ldots, K-1\}$, each state possesses a cluster center θ_k, the model objective is to minimize :

$$\sum_{t=1}^{T} L(v_t, \theta_{s_t}) + \lambda \sum_{t=2}^{T} 1(s_t \neq s_{t-1})$$

Where $L(v_t, \theta_{s_t})$ is the within-cluster squared-distance loss of observation (v_t), the sum of all observations constituting the Within Cluster Sum of Squares $(WCSS)$, and λ the jump penalty. The penalty is what controls regime persitence, if λ is set to 0 the model will act like an ordinary k-means clusterer and ignore temporal order. A smaller λ will allow frequent regime changes and a larger λ will force regimes to represent a more persistent state of the market.

Coordinate descent The jump model is fitted using an alternating coordinate-descent procedure. First, the **state-sequence update** determines the regime sequence that jointly minimizes the clustering loss and regime jump penalties.

Then, the **center update** recomputes the cluster centers using all pooled observations. Because the global jump model is fitted across multiple assets, the state-sequence update is restarted at each asset boundary: the same jump penalty is used within every asset, but no transition penalty is imposed between the last observation of one asset and the first observation of the next, as shown in Fig. 6. The two updates alternate until the regime sequence remains unchanged, the improvement in the objective falls below a specified tolerance, or the maximum number of iterations is reached. The sparse jump model adds an outer **feature-weight update**, in which the weights are obtained by soft-thresholding the feature-wise between-cluster sums of squares subject to the sparsity constraints (explained in the following section, Sparse feature selection).

Fig. 6 illustrates the implemented global Jump Model approach. An alternative method consists of training one jump model per asset and then aggregating the resulting regime labels. Both approaches were investigated and implemented. However, since the resulting label assignments differed by only approximately 5%, the globally trained jump models were retained for the analysis presented in the following sections.

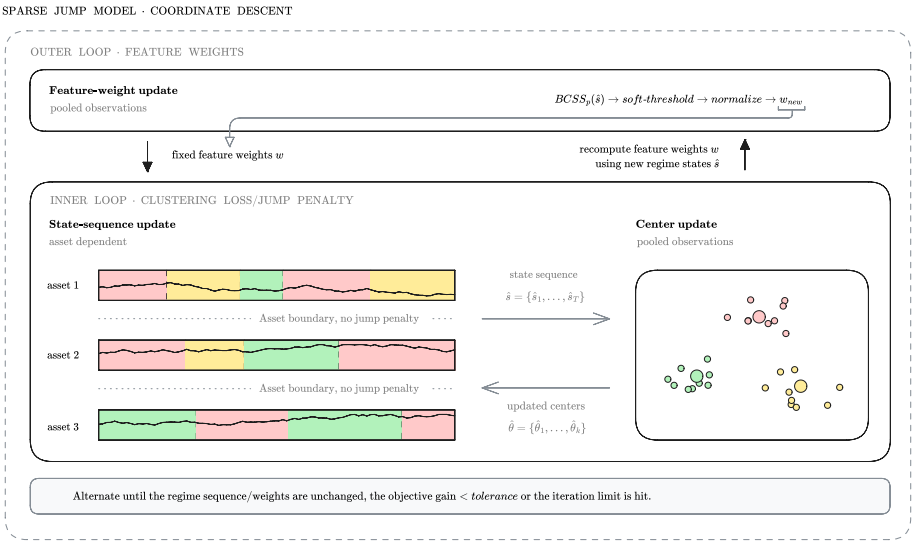


Fig. 6. Global Sparse Jump Model coordinate descent

Sparse feature selection Sparse Jump Models (SJM) are models with the ability to select the most discriminative features, the steps are the following: assign an importance score to every feature, soft-threshold using the κ sparsity parameter and normalize the scores into weights.

Importance score. For each feature p, the model calculates its between-cluster sum of squares (BCSS):

$$BCSS_p(\hat{s}) = \sum_{k=1}^{K} n_k \left(\mu_{kp} - \bar{\mu}_p\right)^2$$

Where K is the number of states, n_k the observations assigned to state k, μ_{kp} the mean of feature p assigned to state k, $\bar{\mu}_p$ the overall mean of feature p and $\hat{s}$ the given state sequence. A large value means that the feature has very different values across states and is therefore useful for distinguishing regimes whereas a small value means that the feature behaves similarly across states and is less relevant.

feature removal. Given the importance score vector obtained in the previous step:

$$v = (BCSS_1(\hat{s}), BCSS_2(\hat{s}), ..., BCSS_p(\hat{s}))$$

and the sparsity parameter κ, the model determines Δ such that:

$$w = \frac{S(v, \Delta)}{||S(v, \Delta)||_2}$$

$$with:$$

$$S(v_p, \Delta) = max(v_p - \Delta, 0)$$

and needs to satisfy the following constraints:

$$||w||_2 = 1 \qquad ||w||_1 \leq \kappa \qquad w_p \geq 0$$

The first one forces the normalization of the scale of the weight vector, the second one that the weights sum up to at most κ (this controls sparsity) and the last one ensures that weights are non-negative. Finally, if the L_1 constraint is already satisfied, $\Delta = 0$.

BIC selection Bayesian Information Criterion is a statistical metric first introduced in 1978 by Schwarz and adapted to sparse jump models in *What Drives Cryptocurrency Returns? A Sparse Statistical Jump Model Approach* by Cortese, Kolm, Lindström (2023) [6], it consists of comparing candidate models by balancing how well the model fits the data and its complexity. For each candidate parameter tuple (λ, κ, K) with λ the jump penalty, κ the sparsity parameter and K the number of states :

$$\text{BIC} = \frac{1}{T}\left\{2\left(\text{BCSS}_T(\bar{\lambda}, \bar{\kappa}, \bar{K}; \mathbf{X}) - \text{BCSS}_T(\lambda, \kappa, K; \mathbf{X})\right) + M\log(T)\right\}$$

Where $\text{BCSS}_T(\bar{\lambda}, \bar{\kappa}, \bar{K}; \mathbf{X})$ and $\text{BCSS}_T(\lambda, \kappa, K; \mathbf{X})$ are the saturated and candidate BCSS after fitting the model. The data matrix containing the features

used to fit the sparse jump model is $\mathbf{X} \in \mathbb{R}^{T \times P}$, with T and P respectively the number of observations and the number of features.

The saturated model. The saturated model is intended to provide the best attainable fit. Its hyperparameters are therefore set to:

$$(\bar{\lambda}, \bar{\kappa}, \bar{K}) = (0, \sqrt{P}, 6)$$

Setting $\bar{\lambda} = 0$ removes the regime-switching penalty, while $\bar{\kappa} = \sqrt{P}$ renders the sparsity constraint inactive, allowing all features to receive positive weights. Finally, $\bar{K} = 6$ specifies a relatively large number of regimes, following the recommendation of Cortese, Kolm, Lindström.

The model fit. The candidate model's fitting loss relative to the saturated model is:

$$2\left(\mathrm{BCSS}_T(\bar{\lambda}, \bar{\kappa}, \bar{K}; \mathbf{X}) - \mathrm{BCSS}_T(\lambda, \kappa, K; \mathbf{X})\right)$$

Because BCSS is maximized, a large $\mathrm{BCSS}_T(\lambda, \kappa, K; \mathbf{X})$ means that the regimes are well separated. A candidate model close to the saturated BCSS has a small fit penalty.

The complexity term. The complexity is defined as :

$$M = K_0|\mathcal{A}_0| + |\mathcal{A}_0|(K - K_0) + K_0\left(|\mathcal{A}| - |\mathcal{A}_0|\right) + \sum_{t=2}^{T} \mathbb{I}_{\{s_t \neq s_{t-1}\}}$$

Where $A = \{p : w_p > 0\}$ is the candidate model active features (with a weight heavier than 0 given by the sparse feature selection), K its number of clusters, and $|A_0|, K_0$ are the reference number of relevant features and number of states. However, $|A_0|$ and K_0 suppose prior knowledge from market states; the selection of these values has a direct impact on the complexity penalty of the information criterion and therefore influences model selection. Assuming no prior knowledge, a solution is to evaluate the best number of clusters on the downstream classification task of M2, by setting $K_0 = K$ and training a model for each K the complexity penalty simplifies to:

$$M = K|\mathcal{A}| + \sum_{t=2}^{T} \mathbb{I}_{\{s_t \neq s_{t-1}\}}$$

Canceling the $|A_0|$ and K_0 terms, but implying a different jump model for every studied K.

FTIC selection Fan-Tang Information Criterion (FTIC) was later published in *Generalized Information Criteria for High-Dimensional Sparse Statistical Jump Models* by Cortese, Kolm, Lindström (2026) [7], it follows the same general trade-off between model fit and complexity as the original BIC, although the fit and complexity terms are redefined.

$$IC = \frac{1}{T}(fit\ term + a_T M)$$

Where in BIC $a_t = log(T)$, in Fan-Tang $a_T = \log(\log(T))\log(p)$ where p is the number of candidate features. FTIC was designed for high-dimensional, low-sample-size settings. In the present low-p, high-T setting, the FTIC penalty coefficient a_T is smaller than $\log(T)$, allowing more complex regime specifications. Both criteria were compared using their respective formulations, and the original BIC was retained as it produced better separation in trend and mean return across regimes on *pre-test* observations.

Jump Models features and fitting The final Sparse Jump Model was fitted on a combined feature set, totaling 54 features, derived from the work of Nystrup, Kolm, Lindström [17], Cortese, Kolm, Lindström [6], and Shu, Yu, Mulvey [26] [25]. Most of these features are either specific to cryptocurrency markets or based on standard technical indicators (OHLCVA) and can therefore be transferred directly. However, several require a market benchmark or a risk-free rate for which no direct equivalent exists in the crypto market, and were consequently adapted.

Daily fit. To obtain persistent regimes and remain consistent with the recent literature, the most intuitive approach was to fit the jump models using the proposed daily features and let the sparse selection mechanism identify the most informative ones. This produces persistent regimes similar to those considered in most of the cited studies. To accommodate different prediction granularities while avoiding look-ahead bias, the resulting daily labels are shifted by one day and then forward-filled over the corresponding time frame.

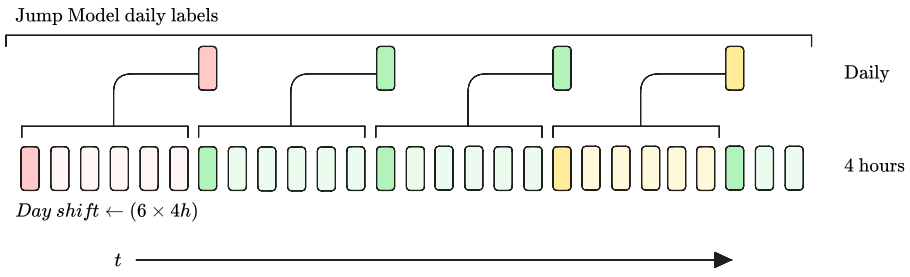


Fig. 7. Shift and forward-fill over granularities

The active return features of Shu, Yu, Mulvey are adapted to cryptocurrencies by measuring each coin's return relative to the CCi30 cryptocurrency market index, rather than an equity factor's return relative to the MSCI USA Index. The original macro-financial variables are retained, with their latest available values carried forward over weekends. Window lengths are interpreted in calendar days to reflect continuous crypto trading. For the excess return features of Shu, Mulvey, the US 3-month Treasury bill rate (DTB3) is converted to a daily risk-free rate using a 365-day basis before being subtracted from each coin's daily return. The 19 features selected by Cortese, Kolm, Lindström are reconstructed using Binance USDT prices and trading activity instead of the original cross-exchange Crypto Asset Lab data. Weekly Google Trends observations are introduced only after the corresponding week has ended and are then carried forward, preventing the use of information before it would have been available.

However, the relevance of such regime definitions becomes less clear when deploying models at finer granularities: for instance, a regime persisting for several months may provide limited information for an hourly trading decision. For this reason, a second approach was also implemented, in which regimes are estimated directly at the native granularity of each prediction horizon.

Native fit. In the native-frequency experiment, all 54 features are computed directly from Binance bars. The original numerical window lengths are retained but interpreted as numbers of bars rather than days, resulting in shorter time horizons. For the active return features, each coin is compared with an equal-weight crypto-market proxy constructed from the available Binance USDT assets, including the target coin. Daily macro-financial variables, such as VIX and Treasury yields, retain their original transformations but are introduced with a one-day delay and carried forward between updates.

For the excess return features, the annual DTB3 rate is converted to a per-bar risk-free rate using a 365-day basis ($365 \times barsperday$), and the original half-lives are interpreted as numbers of bars. The features selected by Cortese, Kolm, Lindström are similarly reconstructed from native Binance closing prices and USDT turnover, with their window lengths also interpreted in bars. Weekly Google Trends observations become available only after the corresponding week has ended, with an additional one-day delay, and are then carried forward until the next update.

5 Methodology

5.1 Feature sets

The M2 model was initially trained on an arbitrarily selected set of 32 features, referred to as the *Deployment baseline*. To study feature behavior across periods and assess their impact on M2 performance, the dataset was expanded to 55 features and partitioned into several subsets: *All Features (55)*, containing all variables; *Deployment baseline (32)*, corresponding to the original selected features; *Basher Sadorsky (23)*, containing the additional features introduced in

the work of Basher, Sadorsky [3]; *OHLCVA_ only (41)*, containing only technical market features; and *XFeatures (14)*, containing only external features.

5.2 Complexity reduction

Considering 20 assets, 8 time granularities, 5 feature sets, 2 trading directions, 6 possible regime configurations, and several candidate M2 models (*TabPFN* [15], *TabICL* [22], *Random Forests*, and *AutoGluon* [10]), the number of possible experimental configurations quickly became prohibitively large. To limit this combinatorial explosion and keep the subsequent analysis tractable, the M2 model was fixed to a Random Forest operating at the 4-hour granularity, as this configuration yielded the most promising results on the *validation* split. Random Forest was selected during preliminary model comparison; however, Extremely Randomized Trees were used to reduce computational cost. Furthermore, the analysis was restricted to M2 Up, as a long-only trading strategy was considered preferable to a long-short setup.

5.3 Walk-forward cross-validation

Originally, a single model was trained using a fixed dataset split:

- *train* from 2024-07-01 to 2025-09-30
- *val* from *train-end* to 2025-12-31
- *test* from *val-end* to 2026-05-31

Although earlier data is available, M2 can only be trained from July 2024 onward because *Kronos base* was trained on the preceding period. This results in approximately 700 usable days, split into 457 training days, 92 validation days, and 151 test days (the limited amount of data was one of the main motivations for introducing smaller granularities and multiple assets). However, a mismatch in trend and volatility was observed between the *train-validation* and *validation-test* periods, making model selection based on a single fixed split unreliable (this mismatch was later confirmed by the jump model regime assignments, *validation* period being mostly bearish by daily regimes). To reduce this dependence on a particular market regime and obtain a more robust estimate of generalization performance, a 5-fold walk-forward cross-validation was introduced.

The dataset is first divided into two subsets, *train+val* and *test*. The *train+val* subset is then partitioned into an initial training period containing 50% of the data and five consecutive validation folds, each representing 10%. For each fold, a cloud of 1,000 Extremely Randomized Trees models is trained using an expanding window and evaluated on the corresponding validation period. Before each validation fold and the final *test* period, all observations present in the 7 bars (forecast horizon) preceding the next *val* or *test* period are purged to prevent target leakage. In addition, when Jump Model regimes are used, a new Jump Model is fitted on each training window so that regime assignments remain causal and no look-ahead bias is introduced.

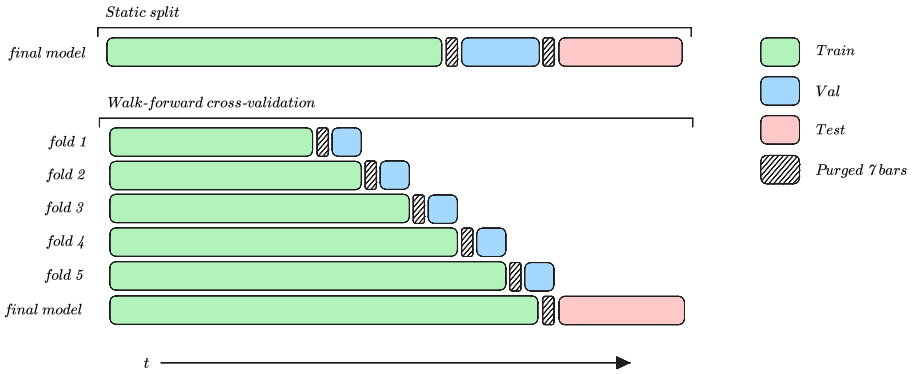


Fig. 8. Walk-forward cross-validation pipeline

6 Experiments

6.1 Jump Models regime assignment

As an illustration of the resulting regime assignments, Fig. 9 and Fig. 10 show the in-sample and causal labels produced by the selected models under the BIC selection criterion for the BTC/USDT trading pair. The jump model can assign regimes either using the full observation window (*in-sample*) or using only the information available up to time t (*causal*). Regimes are internally ordered by the model according to the mean log return of the observations assigned to each state during fitting. For visualization purposes only, however, regime colors are determined using the trend percentage computed over the trailing context window, corresponding to 40 bars at the daily granularity and 90 bars on a 4-hour native fit.

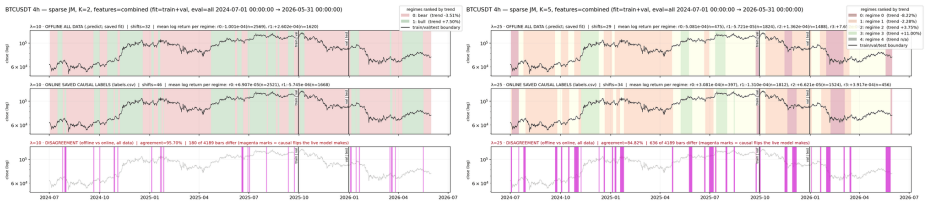


Fig. 9. Daily-fit in sample vs causal labels disagreement, 2 and 5 regimes (BTCUSDT).

Table 1 illustrates the validation mismatch discussed earlier for the two-regime setting. Interestingly, this mismatch largely disappears when using native 4-hour regimes, as the model is able to produce more granular and time-specific assignments. This further supports the idea that an intraday trading strategy may extract limited information from regimes defined in the broader financial

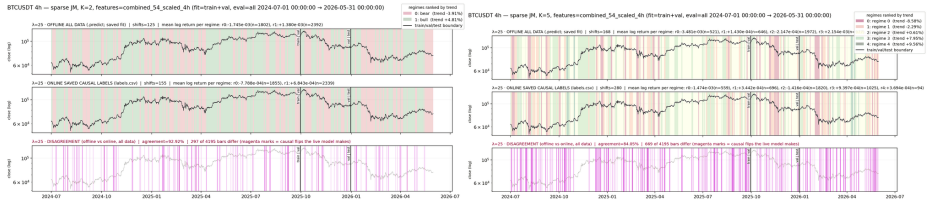


Fig. 10. Native 4h-fit in sample vs causal labels disagreement, 2 and 5 regimes (BTCUSDT).

sense as persistent market states, since such states may not remain sufficiently informative at finer temporal scales.

Table 1. Overall regime occupancy of the two-state Jump Models, for the fixed *train*, *val*, *test* periods and for the walk-forward protocol.

	Daily-fit, forward-filled to 4 h			Native 4h-fit		
Split	Bear (%)	Bull (%)	N	Bear (%)	Bull (%)	N
Fixed splits						
Train	56.94	43.06	54 240	41.63	58.37	54 448
Validation	**90.22**	9.78	11 040	50.53	49.47	11 040
Test	61.99	38.01	17 900	51.52	48.48	18 100
Walk-forward						
Initial train	69.89	30.11	32 180	47.62	52.38	32 608
Pooled validation	59.42	40.58	32 880	42.59	57.41	32 880
Test	62.45	37.55	18 120	51.57	48.43	18 120

Notes: The upper panel uses the fixed *train*, *val* and *test* periods; the lower panel uses the walk-forward protocol, whose initial training period is the 50% burn-in ending on 1 April 2025, whose pooled validation is the union of the five fold intervals ending on 31 December 2025 (each interval using the Jump Model fitted at its own fold cutoff), and whose test assignments use the final Jump Model refitted on all pre-test observations. Bear and bull regimes are identified according to their mean returns over the fitting observation. Occupancy is calculated over the pooled assets (bar assignments for the 20 cryptocurrencies). Differences in N arise from feature availability (different warmup windows).

6.2 Regime features

Adding regimes as input features to the M2 classifier is the most straightforward way to expose the model to this additional information and was therefore the first approach investigated.

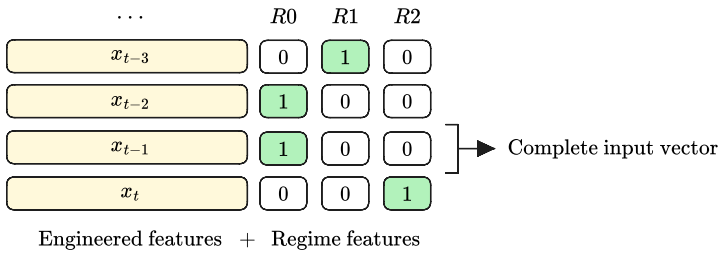


Fig. 11. Regime features

Regimes were represented using one-hot encoding. For example, with $K = 3$, three binary features were concatenated to the original feature set, with exactly one of them equal to 1 for each observation. Fig. 12 and Fig. 13 compare the performance of the cloud of seeded models (M2 extremely randomized trees) over the validation and test periods in mean trade return and precision.

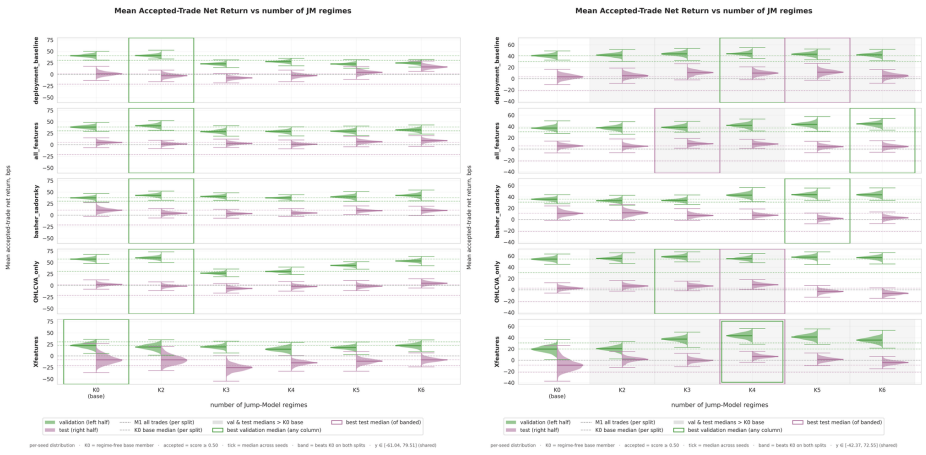


Fig. 12. Mean trade return over feature sets and regime features, daily-fit regimes (left) and 4h regimes (right).

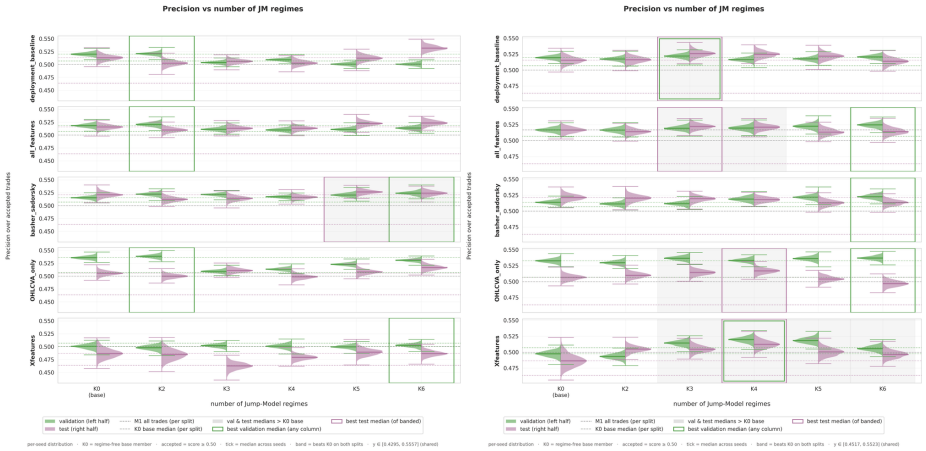


Fig. 13. Precision over feature sets and regime features, daily regimes (left) and 4h regimes (right).

The conclusion of this experiment is particularly clear: regimes defined in the conventional financial sense as persistent market states at the daily scale do not appear to provide meaningful additional information to the model when trading on a 4h window. In contrast, regimes defined natively at the 4-hour granularity have a clear impact on performance. In validation, 23/25 regime-enhanced configurations achieved a higher median mean trade return than their respective baselines, with 16/25 maintaining this improvement on the test set.

However, this comparison is not strictly one-to-one because of differences in information freshness. To avoid look-ahead bias, daily regime features are shifted to the following day before being used at the 4-hour granularity, whereas native 4-hour regimes do not require such a delay. Consequently, the observed improvement may not originate from regime information alone, but rather from the more recent information contained in the isolated feature set summarized by the Jump Model. In this case, information freshness itself may be an important driver of the performance gain.

One question therefore remains: although regime information improves the initial *Deployment baseline*, this benefit is not observed consistently across all feature sets. This raises the possibility that regimes primarily contribute information that is missing from less informative feature subsets. The results for *All Features* partially challenge this interpretation, as three-regime and four-regime configurations still improve performance despite the broader feature coverage. This suggests that the interaction between regime information, feature composition, and information freshness requires further investigation, particularly through a more systematic feature-selection analysis.

Distribution distances To assess the effect of market regimes on the feature and meta-label distributions, the unbiased squared Maximum Mean Discrepancy (MMD) was computed between the corresponding sample distributions (over 55 features), using a Gaussian kernel whose bandwidth follows the median pairwise-distance heuristic estimated once on the pooled rows, over features clipped at 3σ and standardized beforehand.

The distribution distances reported in Table 4 and Table 5, both collected in Section A.1, lead to several conclusions. Meta-label-conditioned feature distributions show limited separation, whereas regimes strongly partition the feature space. Conversely, distributions associated with different regimes remain clearly separated, even when conditioned on the same meta-label. This separation tends to increase as the number of regimes grows, consistent with more distant regimes, such as r_0 and r_5 exhibiting greater distributional differences. It is interesting to note that daily regimes have a larger cross-regime MMD $[0.16, 0.21]$ than their 4h counterpart $[0.14, 0.17]$, thus the daily regimes strongly separate the market feature space but apparently along directions that are less useful for predicting the meta-label. Overall, the regimes primarily capture distinct feature distributions rather than differences in meta-label distributions.

6.3 Regime gating

Another way to exploit regime information is through a regime-specific post-analysis of M2 performance. This is motivated by the observation that M2 behaves significantly differently across regimes, independently of the feature set used. Fig. 14 presents the aggregated performance of 1,000 models across regimes characterized by annualized volatility and the trend over a trailing window of 90 four-hour bars, evaluated using mean accepted trade return and precision. The results indicate that M2 performance is primarily related to market trend and is comparatively less sensitive to volatility, although highly volatile regimes tend to exhibit weaker performance. This pattern is consistent with the behavior observed for M1 (see Section A.2 for complete results of M1 and M2 across regimes). Nevertheless, M2 retains an advantage over random selection when applied to M1 outputs, suggesting that it captures meaningful predictive structure rather than merely reflecting the underlying class distribution.

A complementary question is whether this regime dependence can be turned into a deployable trade filter. Ranking regimes on the *validation* folds alone by median accepted trade net return, and applying the selected regime unchanged to the *test* split, improves both precision and mean accepted trade return in 19 of the 20 configurations examined ($K \in \{2, \ldots, 6\}$, daily-fit and native 4h regimes, walk-forward and fixed protocols), and does so for all 1,000 models of the cloud in each of them. On the walk-forward protocol the gain ranges from +9.8 to +53.1 bps per accepted trade while retaining between 43% and 81% of the accepted windows. The single exception, $K = 3$ on daily-fit regimes, is also the only configuration in which the validation margin between the selected regime and the runner-up falls below 20 bps (4.6 bps, against a minimum of 55.6 bps everywhere else), which suggests that this pattern is consistent with

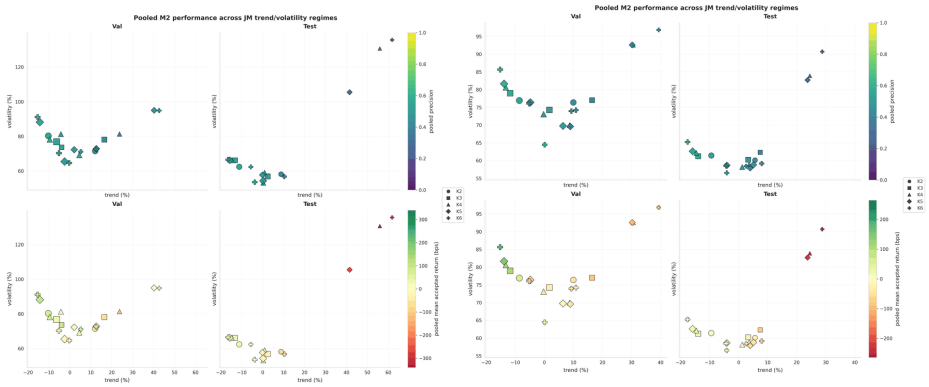


Fig. 14. M2 UP Regime pool map (walk-forward cross-validation), shifted and forward-filled daily regimes (left) and 4h regimes (right).

stronger transfer when the validation margin is larger. Full results are reported in Table 8 and summarised in Fig. 15.

Widening the selection to the joint space of feature sets and regimes does not weaken this conclusion. Validation then selects the *OHLCVA only* set in all ten walk-forward configurations, although that set ranks only fourth of five on test precision, yet gating the resulting model to the validation-preferred regime still adds between +16.3 and +54.1 bps for all 1,000 models (Table 9). Measured as Spearman rank correlation between the validation and test orderings, feature sets transfer at $\rho \approx 0.0$–0.1 whereas regimes transfer at a mean ρ of 0.67, positive in nine of the ten configurations (Table 10). Under distribution shifts, regime ordering transfers substantially more consistently than feature-set ordering, which cautions against relying on validation-only feature-set rankings when designing per-regime feature selection.

These results suggest that regime-specific trade gating and feature selection are promising directions. Under this framework, the model could be optimized specifically for the regime observed at deployment time, naturally leading to regime-dependent feature selection. Feature selection has been a central consideration throughout this project (the reason why multiple feature sets were introduced), and this approach provides a coherent way to integrate regime information across the entire pipeline, from feature construction and selection to model training and trade filtering.

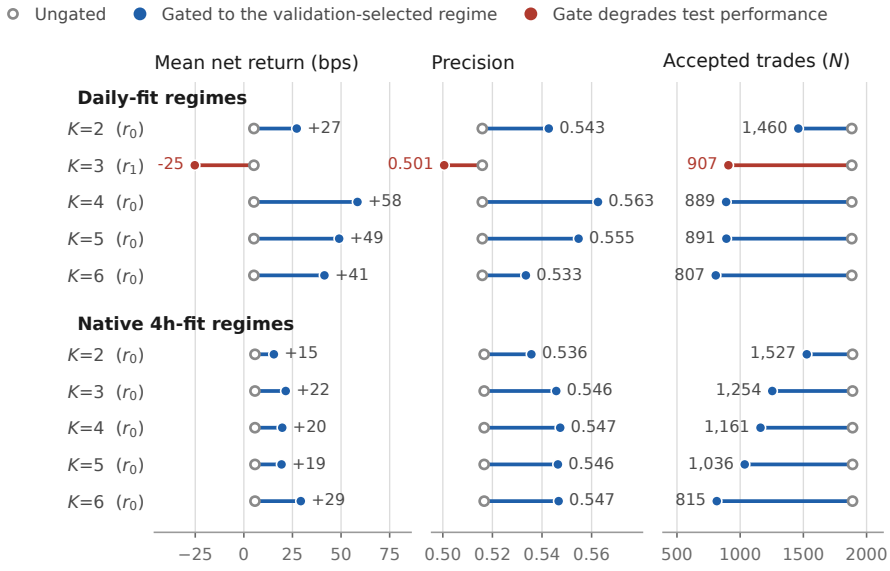


Fig. 15. Ex-ante regime gating on the *test* split, walk-forward protocol: trading every window accepted by the 1,000-model base (without regime features) *all features* cloud (ungated) against trading only those falling in the regime selected on the *validation* folds. Left: mean accepted trade net return, net of the 0.2% transaction fee. Centre: precision over accepted trades. Right: number of accepted trades, giving the coverage the gate retains out of the 1,883 (daily-fit) and 1,889 (native 4h) windows accepted ungated. Values are medians across the 1,000 models. The gate raises both metrics in nine of the ten configurations; the exception, $K = 3$ under daily-fit regimes, is the only one whose validation selection margin fell below 20 bps.

7 Complementary experiments

The two analyses reported in this section were conducted alongside the regime study but answer separate questions: which features remain informative once the distribution shifts, and which single seed to retain from a model cloud at deployment time. Both draw on the regime machinery introduced above without forming part of the regime argument itself, and are reported apart from it so as not to dilute the main results.

7.1 M2 feature analysis

Feature selection in a high volatility and ever-changing market is particularly challenging. Because simultaneous distributional shifts between input features and their associated labels may happen over time, some features can prove to be informative during a time period and detrimental in another. Therefore, selecting

features based on a defined *validation* period could lead to poor performance once the model is deployed.

Fig. 16 illustrates this issue by reporting the signed ROC-AUC of each feature across the fixed *train*, *validation*, and *test* periods. A feature exhibiting a positive association during the training and validation periods but reversing sign in the test period provides an indication of concept drift (or high sampling noise). For instance, the volume coefficient of variation (*vol_cv*) would likely have been selected based on the *validation* period, as it clearly separates correct from incorrect M1 predictions. However, this relationship reverses during the test period. This behavior is not isolated, as many features fail to maintain consistent separability across time periods.

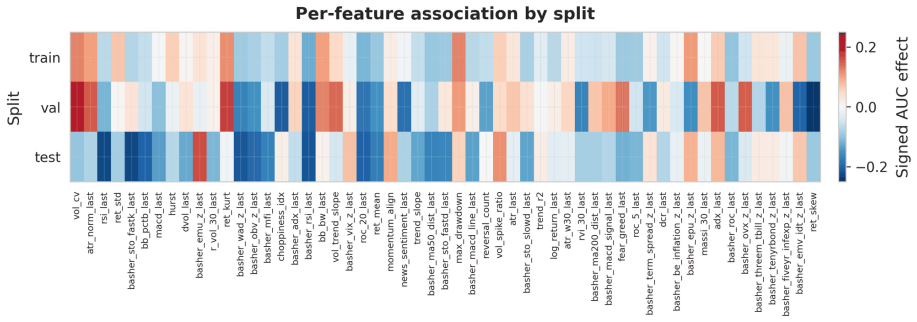


Fig. 16. Signed ROC-AUC by feature and split.

Under these conditions, generic recursive sequential feature-selection or elimination procedures were unlikely to provide robust results, as they typically rely on performance degradation after adding or removing features (in this setting, precision is a more appropriate selection metric than accuracy, since only positive predictions correspond to accepted trades).

Given the previous results showing that regimes strongly partition the feature space, together with the observation that M2 exhibits a genuine edge in specific regimes, the working hypothesis became that performing feature selection on a per-regime basis could reduce the impact of distribution shifts by operating on more homogeneous subsets of the data. Fig. 17 illustrates the resulting sequential selection procedures. An initial search is performed using 30 ExtraTrees estimators to identify a greedy feature-selection path maximizing Average Precision (AP). The search stops when no further feature removal improves the best AP obtained so far. Each subset identified along this path is then evaluated more robustly using 100 independently seeded models. Finally, the difference in AP between the selected subset and the *All Features* baseline is measured on the *test* split to assess whether the selected representation transfers out of sample. AP was chosen to reduce sensitivity to biases arising from a miscalibrated model and because mean trade return can be disproportionately influenced by extreme

outliers. However, this choice is not definitive, since return ultimately remains the target metric in a trading setting.

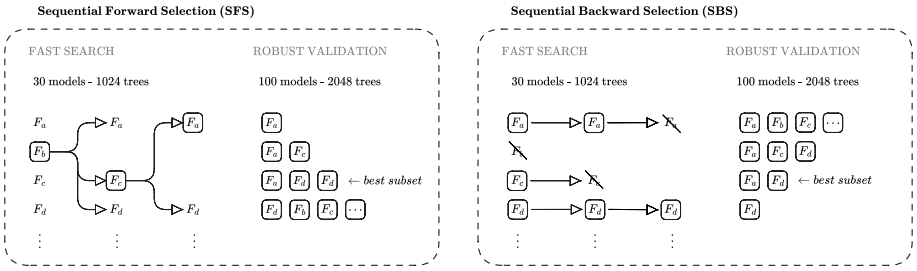


Fig. 17. SFS and SBS pipelines

Sequential Backward Selection was chosen over forward selection, as removing detrimental features from an initially complete feature set appeared more appropriate than progressively reconstructing the feature space. The procedure was therefore applied independently within each regime.

Table 2. Walk-forward: out-of-sample performance of sequential backward selection (SBS) over the *All features* baseline, for daily-fit and native-4h regimes.

Regime	Features	CV AP	Test AP	Test Precision	Δ Coverage	Δ Net Return
Daily-fit regimes						
r_0 (bear)	55 → 36	0.6228 → 0.6784	0.5900 → 0.5885	0.5450 → 0.5424	−3.56 pp	−0.31 bps
r_1 (bull)	55 → 25	0.4868 → 0.5827	0.3947 → 0.3650	0.4008 → 0.3748	+2.46 pp	−6.19 bps
Native-4h regimes						
r_0 (bear)	55 → 36	0.6068 → 0.6752	0.5855 → 0.5796	0.5300 → 0.5317	−4.96 pp	+3.63 bps
r_1 (bull)	55 → 26	0.4863 → 0.5765	0.4322 → 0.3587	0.4373 → 0.3560	−7.53 pp	−67.61 bps

As shown in Table 2, the improvements obtained during cross-validation do not consistently transfer to the out-of-sample *test* period, even when feature selection is performed within individual regimes. In the bearish regime, the selected subset provides only marginal changes in AP and precision. In the bullish regime, however, the substantial cross-validation improvement reverses out of sample, with clear deterioration across AP, precision, and net return.

These results suggest that regime conditioning alone is insufficient to make generic sequential feature selection robust to distribution shifts and that the feature selection may be overfitting noise. However, it is interesting to note that removing one-third of the features on the regimes where M2 achieves the strongest performance (bearish) does not degrade the out-of-sample results, suggesting that the 19 removed features provided limited incremental value conditional on the retained feature set and model. Nevertheless, given the large performance

differences previously observed across feature sets, feature selection remains an important direction to further study.

Interesting note

Even though SBS did not provide reliable out-of-sample improvements, it is interesting to note that the selected feature subsets for r_0 and r_1 are of comparable size. Moreover, both the native 4-hour and daily-fit regime configurations retain many of the same features in r_0, the regime in which M2 achieves its strongest performance.

Table 3. Pairwise overlap between SBS-selected feature subsets across regimes and Jump Model fittings.

Comparison	Set A	Set B	Shared	Overlap/A	Overlap/B	Jaccard
Same regime, different JM fit						
Bear regime	Native r_0 (36)	Daily r_0 (36)	31	86.1%	86.1%	75.6%
Bull regime	Native r_1 (26)	Daily r_1 (25)	15	57.7%	60.0%	41.7%
Different regimes, same JM fit						
Native 4h	Native r_0 (36)	Native r_1 (26)	18	50.0%	69.2%	40.9%
Daily-fit	Daily r_0 (36)	Daily r_1 (25)	18	50.0%	72.0%	41.9%

Note: Parentheses report the number of features in each selected subset. Overlap/A and Overlap/B denote the number of shared features divided by the size of Set A and Set B, respectively. Jaccard similarity is the intersection divided by the union.

7.2 Seed selection

Throughout this work, clouds of random seeds have been used to robustly assess and compare M2 performance. However, deployment ultimately requires selecting a single seed. Initially, selecting this seed solely according to *validation* performance appeared suboptimal, as the distributional mismatch between the *train*, *validation*, and *test* periods resulted in poor transferability.

The proposed solution was therefore to use regime information to compute a regime-balanced precision score and select the seed exhibiting the most robust performance across market states. This approach produced promising results on the fixed split (see Section A.4), but transferred poorly to the walk-forward cross-validation setting, where the distributional mismatch between folds was substantially reduced.

Fig. 22 and Fig. 23 (see Section A.5) illustrate the seeds selected from the best-performing model clouds identified during the cross-validation experiments. These correspond to the configurations that would have been retained based on the cross-validation results. Mean accepted trade return is used as the primary selection criterion rather than precision, as precision alone does not capture differences in profitability between accepted trades.

As shown in the figures, using regime-balanced precision on the *validation* period provides little to no additional robustness on the *test* period, suggesting

that there is no evidence that seed tuning adds out-of-sample value. Therefore, a fixed pre-specified seed would avoid another selection degree of freedom.

8 Future steps

8.1 On regime as a feature

One of the main unresolved questions is why the native 4-hour Jump Model regime features improve the M2 classifier. Two experiments are therefore proposed to disentangle the effect of regime structure from that of the underlying feature information. First, the Jump Model can be trained on exactly the same feature set as M2. If the resulting regime features still improve M2 performance, this would suggest that the clustering and temporal-persistence properties of the Jump Model capture useful structure that M2 cannot directly exploit, supporting regimes as a meaningful additional signal. Second, M2 can be trained on the union of its original features and the features used by the Jump Model. If adding regime features no longer improves performance in this setting, the previous gains would likely originate from the additional information contained in the Jump Model feature set rather than from the regime representation itself. Conversely, if regime features continue to provide an improvement, this would indicate that the regime abstraction carries information beyond the raw features from which it is derived.

8.2 On regime gating

Recent results indicate that M2 performs particularly well in bearish regimes across both the *validation* and *test* periods. A straightforward strategy would therefore be to filter out trades associated with regimes in which the model performs poorly. As observed during the internship experiments, however, training M2 exclusively on a given regime and deploying it only within that regime does not materially improve or degrade performance on the corresponding trades, suggesting that the limitation may lie upstream of regime-specific M2 training (M1 forecaster), although reduced sample size and model capacity remain alternative explanations.

Regime information also localizes the M2 edge to better-defined subsets of trades, which makes per-regime feature selection a natural next step. The obstacle is that validation cannot rank feature sets: their ordering does not transfer to the *test* split ($\rho \approx 0.0$–0.1, against a mean $\rho = 0.67$ for regimes, Table 10). A usable procedure therefore needs a criterion beyond the validation score, such as stability of the selected subset across folds or a per-regime restriction of the candidate space. Combined with regime-specific trade filtering, this would carry regime information through the entire pipeline and provide a coherent basis for a genuinely regime-aware classifier.

9 Conclusion

Market regimes were investigated as a means of mitigating distribution shift in a two-stage cryptocurrency architecture. They partition the feature space sharply while carrying little direct information about the meta-label, which is why they diagnose and localize M2 performance far better than they improve it. Their most useful role proved to be trade filtering: restricting M2 to the regime preferred on *validation* raised both precision and mean accepted trade return in nearly every configuration tested, provided the validation ranking was decisive rather than near-tied.

Regime-conditioned feature and seed selection, by contrast, did not transfer out of sample, and a joint-selection comparison indicates why: across the *validation–test* boundary the regime ordering survives whereas the feature-set ordering does not. Market regimes are therefore useful for diagnosing, localizing and filtering model performance, but insufficient on their own to resolve distribution shift, and a longer out-of-sample record would be needed to establish how much of the observed gain is durable.

This internship has been a very positive and enriching learning experience. I had the opportunity to meet several passionate Ph.D. students and immerse myself in their working environment, which gave me a better understanding of how the research process works. During the weekly meetings, I observed discussions on how to structure a research paper, how to address reviewer feedback after submission, and how to properly evaluate and interpret results.

One of the main lessons I learned is that a significant part of research takes place before any implementation begins. It involves carefully reviewing and understanding the state of the art, identifying relevant research questions, and determining what makes an idea promising or less compelling from a research perspective. Beyond the research aspect, coming to Japan was a once-in-a-lifetime experience that allowed me to discover a new working culture, adapt to a different environment, and grow both professionally and personally. To conclude, it was a pleasure to work in such a stimulating environment.

A Annexes

A.1 Distribution distances across regimes

Let r_x denote the samples assigned to regime x, and let y_0, y_1 denote the samples with meta-labels 0 and 1, respectively. The notation $d_{MMD}(a, b)$ represents the MMD between sample sets a and b.

Table 4. MMD between 4h meta-label and regime-conditioned distributions, using the daily-fit Jump Model. The last column averages the 95% calendar-day cluster-bootstrap interval over the K-columns in which the row is defined.

Regimes	**2**	**3**	**4**	**5**	**6**	**Avg 95 % CI bounds**
$d_{\mathrm{MMD}}(y_0, y_1)$	0.004	0.004	0.004	0.004	0.004	[−0.002, 0.006]
$\mathrm{mean}_x\, d_{\mathrm{MMD}}(r_x \cap y_0, r_x \cap y_1)$	0.004	0.007	0.005	0.006	0.005	[−0.002, 0.006]
$d_{\mathrm{MMD}}(r_0 \cap y_0, r_0 \cap y_1)$	0.005	0.009	0.009	0.009	0.010	[−0.003, 0.011]
$d_{\mathrm{MMD}}(r_1 \cap y_0, r_1 \cap y_1)$	0.003	0.005	0.004	0.003	0.003	[−0.004, 0.004]
$d_{\mathrm{MMD}}(r_2 \cap y_0, r_2 \cap y_1)$		0.009	0.001	0.003	0.004	[−0.007, 0.006]
$d_{\mathrm{MMD}}(r_3 \cap y_0, r_3 \cap y_1)$			0.007	0.005	0.002	[−0.008, 0.006]
$d_{\mathrm{MMD}}(r_4 \cap y_0, r_4 \cap y_1)$				0.009	0.004	[−0.008, 0.008]
$d_{\mathrm{MMD}}(r_5 \cap y_0, r_5 \cap y_1)$					0.008	[−0.012, 0.011]
$\mathrm{mean}_{x \neq x'}\, d_{\mathrm{MMD}}(r_x \cap y_1, r_{x'} \cap y_1)$	0.172	0.204	0.222	0.215	0.228	[0.181, 0.228]
$\mathrm{mean}_{x \neq x'}\, d_{\mathrm{MMD}}(r_x \cap y_0, r_{x'} \cap y_0)$	0.143	0.158	0.201	0.185	0.201	[0.152, 0.196]
$\mathrm{mean}_{x \neq x'}\, d_{\mathrm{MMD}}(r_x, r_{x'})$	0.156	0.178	0.210	0.198	0.213	[0.170, 0.208]

Table 5. MMD between 4h meta-label and regime-conditioned distributions, using the native 4h-fit Jump Model. The last column averages the 95% calendar-day cluster-bootstrap interval over the K-columns in which the row is defined.

Regimes	**2**	**3**	**4**	**5**	**6**	**Avg 95 % CI bounds**
$d_{\mathrm{MMD}}(y_0, y_1)$	0.004	0.004	0.004	0.004	0.004	[−0.001, 0.006]
$\mathrm{mean}_x\, d_{\mathrm{MMD}}(r_x \cap y_0, r_x \cap y_1)$	0.004	0.005	0.006	0.005	0.005	[−0.001, 0.005]
$d_{\mathrm{MMD}}(r_0 \cap y_0, r_0 \cap y_1)$	0.005	0.006	0.007	0.008	0.010	[−0.004, 0.009]
$d_{\mathrm{MMD}}(r_1 \cap y_0, r_1 \cap y_1)$	0.002	0.001	0.002	0.002	0.002	[−0.003, 0.002]
$d_{\mathrm{MMD}}(r_2 \cap y_0, r_2 \cap y_1)$		0.007	0.005	0.003	0.002	[−0.004, 0.006]
$d_{\mathrm{MMD}}(r_3 \cap y_0, r_3 \cap y_1)$			0.009	0.004	0.002	[−0.005, 0.007]
$d_{\mathrm{MMD}}(r_4 \cap y_0, r_4 \cap y_1)$				0.009	0.005	[−0.008, 0.010]
$d_{\mathrm{MMD}}(r_5 \cap y_0, r_5 \cap y_1)$					0.008	[−0.010, 0.012]
$\mathrm{mean}_{x \neq x'}\, d_{\mathrm{MMD}}(r_x \cap y_1, r_{x'} \cap y_1)$	0.152	0.158	0.190	0.178	0.185	[0.149, 0.189]
$\mathrm{mean}_{x \neq x'}\, d_{\mathrm{MMD}}(r_x \cap y_0, r_{x'} \cap y_0)$	0.133	0.127	0.153	0.149	0.155	[0.124, 0.156]
$\mathrm{mean}_{x \neq x'}\, d_{\mathrm{MMD}}(r_x, r_{x'})$	0.142	0.140	0.168	0.160	0.168	[0.139, 0.169]

A.2 Results of M1 and M2 across regimes

Table 6. M1 and M2 performance across daily-fit Jump Model regimes, shifted by one day and forward-filled onto the 4h grid.

K	Regime	Model	Validation			Test		
			N	Precision	Return (bps)	N	Precision	Return (bps)
2	r_0	M1	3,697	**0.539**	**69.0**	1,924	**0.513**	**9.0**
		M2	2,978	**0.555**	**83.4**	1,460	**0.543**	**27.2**
	r_1	M1	2,530	0.460	-25.3	959	0.365	-81.5
		M2	1,040	0.413	-89.8	423.5	0.423	-70.8
3	r_0	M1	4,043	**0.529**	51.8	1,272	**0.501**	**14.5**
		M2	2,818	0.542	58.9	975	**0.530**	**33.6**
	r_1	M1	861	0.524	**59.0**	1,611	0.435	-49.2
		M2	659	**0.543**	**86.1**	907	0.501	-25.2
	r_2	M1	1,323	0.430	-52.1	–	–	–
		M2	543	0.363	-124.3	–	–	–
4	r_0	M1	1,352	0.479	33.5	1,149	**0.526**	**32.1**
		M2	800	0.466	2.9	889	**0.563**	**58.4**
	r_1	M1	2,541	**0.540**	**65.6**	569	0.434	-55.2
		M2	1,973	**0.564**	**96.1**	319	0.500	-44.7
	r_2	M1	1,702	0.533	32.4	1,111	0.428	-42.7
		M2	762	0.545	22.7	625	0.482	-16.3
	r_3	M1	632	0.364	-120.1	54†	0.204	-349.9
		M2	486	0.377	-111.3	51†	0.216	-352.7
5	r_0	M1	2,090	**0.544**	**99.0**	1,093	**0.520**	**24.1**
		M2	1,811	**0.555**	**108.3**	891	**0.555**	**49.0**
	r_1	M1	2,134	0.495	-3.0	1,255	0.427	-40.7
		M2	1,230	0.511	-6.0	687	0.481	-18.3
	r_2	M1	724	0.493	10.2	442	0.468	-30.9
		M2	466	0.502	15.1	242	0.533	-14.0
	r_3	M1	910	0.485	-13.8	–	–	–
		M2	331	0.424	-88.4	–	–	–
	r_4	M1	369	0.444	-11.5	93†	0.280	-240.5
		M2	181	0.418	-64.4	63†	0.274	-284.0
6	r_0	M1	1,717	**0.556**	**127.0**	977	0.501	**17.7**
		M2	1,504	**0.563**	**133.5**	807	0.533	**41.4**
	r_2	M1	1,579	0.470	-44.4	353	**0.518**	-11.3
		M2	1,067	0.499	-16.3	225	**0.555**	-0.1
	r_1	M1	1,155	0.523	38.7	937	0.494	-10.7
		M2	656	0.523	0.5	619	0.533	15.6
	r_3	M1	572	0.500	20.6	571	0.336	-85.0
		M2	344	0.515	26.7	187	0.402	-99.7
	r_4	M1	881	0.486	-11.8	–	–	–
		M2	289	0.418	-86.5	–	–	–
	r_5	M1	323	0.443	-9.0	45†	0.222	-345.7
		M2	157	0.399	-81.4	44†	0.227	-336.3

Notes: Regimes are ordered within each K from the lowest to the highest mean trailing-window trend, where the trend of a bar is the cumulative return over the trailing M2 window and the mean is taken over the *validation* and *test* bars pooled. Pooling the two splits fixes the row order only, and no reported value depends on it. M1 use all available trades. M2 uses trades accepted at a probability threshold of 0.5 by the 1,000-model base *all features* cloud, trained without regime features and conditioned ex post on regime assignments. M2 values, including N, are medians across models. Returns are net of the 0.2% transaction fee and reported in basis points. Bold values mark the best sufficiently supported regime for each model, K, and split. Underlining marks the higher precision between M1 and M2 for each regime and split; ties are both underlined. A † denotes fewer than 100 trades and is excluded from boldface highlighting.

Table 7. M1 and M2 performance across native 4h-fit Jump Model regimes.

K	Regime	Model	Validation			Test		
			N	Precision	Return (bps)	N	Precision	Return (bps)
2	r_0	M1	3,747	**0.539**	**55.4**	1,954	**0.510**	**4.9**
		M2	3,151	**0.544**	**65.0**	1,527	**0.536**	**15.4**
	r_1	M1	2,480	0.458	-6.6	929	0.367	-75.7
		M2	904	0.422	-58.5	362	0.436	-35.4
3	r_0	M1	2,558	**0.568**	**114.2**	1,398	**0.542**	**21.2**
		M2	2,307	**0.564**	**111.7**	1,254	**0.546**	**21.5**
	r_1	M1	2,553	0.476	-30.5	1,129	0.422	-29.7
		M2	1,397	0.472	-54.9	527	0.471	-15.2
	r_2	M1	1,116	0.438	-20.7	356	0.287	-160.1
		M2	350	0.380	-81.7	107	0.402	-76.2
4	r_0	M1	2,226	**0.575**	**121.8**	1,270	**0.548**	**21.7**
		M2	2,112	**0.572**	**120.8**	1,161	**0.547**	**19.7**
	r_1	M1	2,542	0.485	-25.8	1,172	0.438	-17.2
		M2	1,509	0.484	-40.6	575	0.484	-0.2
	r_2	M1	1,001	0.447	-4.1	330	0.306	-138.0
		M2	267	0.378	-57.5	111	0.459	-6.7
	r_3	M1	458	0.432	-22.3	111	0.243	-204.4
		M2	169	0.335	-154.9	42†	0.263	-272.1
5	r_0	M1	1,894	**0.603**	**157.7**	1,097	**0.544**	**20.0**
		M2	1,818	**0.598**	**154.3**	1,036	**0.546**	19.4
	r_1	M1	1,817	0.462	-45.6	708	0.482	16.7
		M2	1,151	0.457	-66.9	497	0.507	**28.9**
	r_2	M1	1,234	0.488	-6.9	681	0.423	-43.9
		M2	704	0.480	-31.5	226	0.480	-42.6
	r_3	M1	896	0.452	2.6	292	0.308	-131.5
		M2	267	0.375	-55.0	92†	0.411	-55.0
	r_4	M1	386	0.438	-47.5	105	0.200	-250.2
		M2	115	0.388	-123.8	37†	0.294	-243.5
6	r_0	M1	1,531	**0.607**	**180.3**	854	**0.542**	**28.5**
		M2	1,480	**0.601**	**177.3**	815	0.547	**29.3**
	r_2	M1	1,779	0.457	-54.1	694	0.480	11.7
		M2	1,165	0.447	-79.8	486	0.503	21.4
	r_1	M1	938	0.571	45.9	473	0.516	-10.2
		M2	714	0.583	45.7	316	**0.553**	-7.9
	r_3	M1	956	0.442	-9.4	572	0.371	-72.4
		M2	430	0.378	-85.9	173	0.420	-67.9
	r_4	M1	845	0.434	-20.7	230	0.317	-131.8
		M2	202	0.390	-28.0	77†	0.455	-21.9
	r_5	M1	178	0.500	-29.3	60†	0.200	-279.0
		M2	65†	0.431	-116.3	22†	0.150	-338.5

Notes: Regimes are ordered within each K from the lowest to the highest mean trailing-window trend, where the trend of a bar is the cumulative return over the trailing M2 window and the mean is taken over the *validation* and *test* bars pooled. Pooling the two splits fixes the row order only, and no reported value depends on it. M1 use all available trades. M2 uses trades accepted at a probability threshold of 0.5 by the 1,000-model base *all features* cloud, trained without regime features and conditioned ex post on regime assignments. M2 values, including N, are medians across models. Returns are net of the 0.2% transaction fee and reported in basis points. Bold values mark the best sufficiently supported regime for each model, K, and split. Underlining marks the higher precision between M1 and M2 for each regime and split; ties are both underlined. A † denotes fewer than 100 trades and is excluded from boldface highlighting.

A.3 Regime gating results

Table 8. Ex-ante regime gating: performance on the *test* split of restricting M2 trades to regimes selected on the *validation* folds alone.

K	Rule	Gate	Validation basis		Test			
			Ret. (bps)	Margin	Cov.	Δ Precision	Δ Return (bps)	Win
Daily-fit regimes, walk-forward cross-validation								
2	A	r_0	+83.4	158	0.78	**+0.027**	**+22.0**	1.00
	B	r_0			0.78	**+0.027**	**+22.0**	1.00
3	A	r_1	+62.1	5	0.48	-0.015	-30.2	0.00
	B	r_0, r_1			1.00	+0.000	+0.0	0.00
4	A	r_0	+100.5	90	0.47	**+0.047**	**+53.1**	1.00
	B	r_0, r_1			0.64	**+0.030**	**+26.1**	1.00
5	A	r_0	+106.5	99	0.47	**+0.039**	**+43.8**	1.00
	B	r_0, r_1			0.84	**+0.007**	**+14.5**	1.00
6	A	r_0	+94.1	62	0.43	**+0.018**	**+36.1**	1.00
	B	r_0, r_1, r_2			0.88	**+0.021**	**+20.9**	1.00
Daily-fit regimes, fixed splits								
2	A	r_0	+20.6	–	0.77	**+0.025**	**+20.3**	1.00
	B	r_0			0.77	**+0.025**	**+20.3**	1.00
3	A	r_0	+20.6	–	0.50	**+0.013**	**+29.3**	1.00
	B	r_0			0.50	**+0.013**	**+29.3**	1.00
4	A	r_0	+39.8	72	0.46	**+0.045**	**+51.5**	1.00
	B	r_0			0.46	**+0.045**	**+51.5**	1.00
5	A	r_0	+44.3	93	0.45	**+0.035**	**+41.4**	1.00
	B	r_0			0.45	**+0.035**	**+41.4**	1.00
6	A	r_0	+34.1	56	0.40	**+0.015**	**+35.5**	1.00
	B	r_0			0.40	**+0.015**	**+35.5**	1.00
Native 4h regimes, walk-forward cross-validation								
2	A	r_0	+68.0	133	0.81	**+0.019**	**+9.8**	1.00
	B	r_0			0.81	**+0.019**	**+9.8**	1.00
3	A	r_0	+105.0	169	0.66	**+0.029**	**+15.8**	1.00
	B	r_0			0.66	**+0.029**	**+15.8**	1.00
4	A	r_0	+109.1	160	0.61	**+0.031**	**+14.1**	1.00
	B	r_0			0.61	**+0.031**	**+14.1**	1.00
5	A	r_0	+129.2	147	0.55	**+0.030**	**+13.7**	1.00
	B	r_0			0.55	**+0.030**	**+13.7**	1.00
6	A	r_0	+142.6	80	0.43	**+0.030**	**+23.5**	1.00
	B	r_0, r_1			0.60	**+0.032**	**+13.2**	1.00
Native 4h regimes, fixed splits								
2	A	r_0	+26.1	96	0.79	**+0.018**	**+8.1**	1.00
	B	r_0			0.79	**+0.018**	**+8.1**	1.00
3	A	r_0	+46.7	147	0.62	**+0.031**	**+16.3**	1.00
	B	r_0			0.62	**+0.031**	**+16.3**	1.00
4	A	r_0	+48.5	152	0.57	**+0.034**	**+15.5**	1.00
	B	r_0			0.57	**+0.034**	**+15.5**	1.00
5	A	r_0	+60.8	105	0.50	**+0.029**	**+13.9**	1.00
	B	r_0			0.50	**+0.029**	**+13.9**	1.00
6	A	r_0	+84.6	92	0.39	**+0.027**	**+23.0**	1.00
	B	r_0			0.39	**+0.027**	**+23.0**	1.00

Notes: Regimes are ranked on the *validation* folds only; the *test* split never enters the selection. **Rule A** keeps the single regime with the highest median accepted trade net return on validation among those carrying at least 100 accepted trades; **Rule B** keeps every such regime whose validation return is positive. The ungated reference accepts every window at a probability threshold of 0.5 from the 1,000-model base *all features* cloud, trained without regime features. "Ret." and "Margin" report the validation return of the regime selected by Rule A and its lead over the runner-up regime, that is, the evidence on which the selection rests; "–" marks the configurations in which a single regime was eligible. Coverage is the share of ungated accepted trades retained. Δ Precision and Δ Return are median per-model paired differences against the ungated strategy, and "Win" the fraction of the 1,000 models for which the rule raises mean net return. Returns are net of the 0.2% transaction fee and reported in basis points. Bold marks the configurations in which the rule improves mean net return.

Table 9. Joint selection of the feature set and the regime on the *validation* folds, evaluated on the *test* split (walk-forward cross-validation).

K	Gate	Validation margins		Ungated		Gated		
		Δ_{set}	Δ_{reg}	Prec.	Ret.	Prec.	Ret.	Δ Return
Daily-fit regimes								
2	r_0	+8	+118	0.506	+2.5	0.538	+29.1	**+26.6**
3	r_0	+31	+31	0.506	+2.5	0.528	+38.3	**+35.8**
4	r_0	+42	+117	0.506	+2.5	0.545	+56.6	**+54.1**
5	r_0	+39	+103	0.506	+2.5	0.542	+49.6	**+47.2**
6	r_0	+50	+91	0.506	+2.5	0.526	+46.5	**+44.0**
Native 4h regimes								
2	r_0	+15	+111	0.507	+2.9	0.532	+19.2	**+16.3**
3	r_0	+19	+149	0.507	+2.9	0.556	+35.0	**+32.1**
4	r_0	+23	+148	0.507	+2.9	0.561	+34.2	**+31.3**
5	r_0	+21	+143	0.507	+2.9	0.568	+38.6	**+35.7**
6	r_0	+17	+81	0.507	+2.9	0.567	+49.3	**+46.4**

Notes: The selected pair is the (feature set, regime) combination maximising the median accepted trade net return on the *validation* folds, searched over the five base feature sets and all regimes. Validation selected the *OHLCVA only* set in all ten configurations, although that set ranks only fourth of five on test precision (0.506 against 0.516 for *all features*). Δ_{set} and Δ_{reg} are the validation leads of the selected pair over the best candidate using a different feature set and over the best candidate using a different regime, respectively. "Ungated" reports the selected feature set traded over all regimes, "Gated" the same models restricted to the selected regime. Gating raised mean net return for all 1,000 models in every configuration. The transfer of the validation ordering to the test split is quantified in Table 10. Returns are net of the 0.2% transaction fee and reported in basis points.

Table 10. Transfer of the *validation* ordering to the *test* split, measured by Spearman rank correlation.

Ranking	**Regimes**	K	n	**Excluded**	ρ	p
Regimes	Daily-fit	2	2	–	+1.00	1.000
		3	2	r_2	−1.00	1.000
		4	4	–	+0.80	0.333
		5	4	r_3	+0.80	0.333
		6	5	r_4	+1.00	0.017
	Native 4h	2	2	–	+1.00	1.000
		3	3	–	+1.00	0.333
		4	4	–	+1.00	0.083
		5	5	–	+0.40	0.517
		6	6	–	+0.66	0.175
Feature sets	Daily-fit	–	5	–	+0.10	0.950
	Native 4h	–	5	–	+0.00	1.000

Notes: Candidates are ranked by median accepted trade net return on *validation*, ranked again on *test*, and the two orderings compared by Spearman ρ (+1 identical, −1 reversed). Candidates are the regimes of each configuration (upper panel) or the five base feature sets (lower panel); walk-forward protocol, base (without regime features) *all features* cloud at threshold 0.5. n is the number of candidates ranked and "Excluded" lists regimes absent from the test split. p is two-sided and computed by exact permutation over all $n!$ orderings. Mean regime ρ is +0.67, or +0.81 over the seven configurations with $n > 2$ ($n = 2$ admits only $\rho = \pm 1$, hence $p = 1$). Only one configuration reaches $p < 0.05$: the evidence is the regime ordering being positive in nine of ten configurations against feature-set orderings indistinguishable from zero, not any single correlation.

A.4 Results on the fixed *train*, *val*, *test* splits

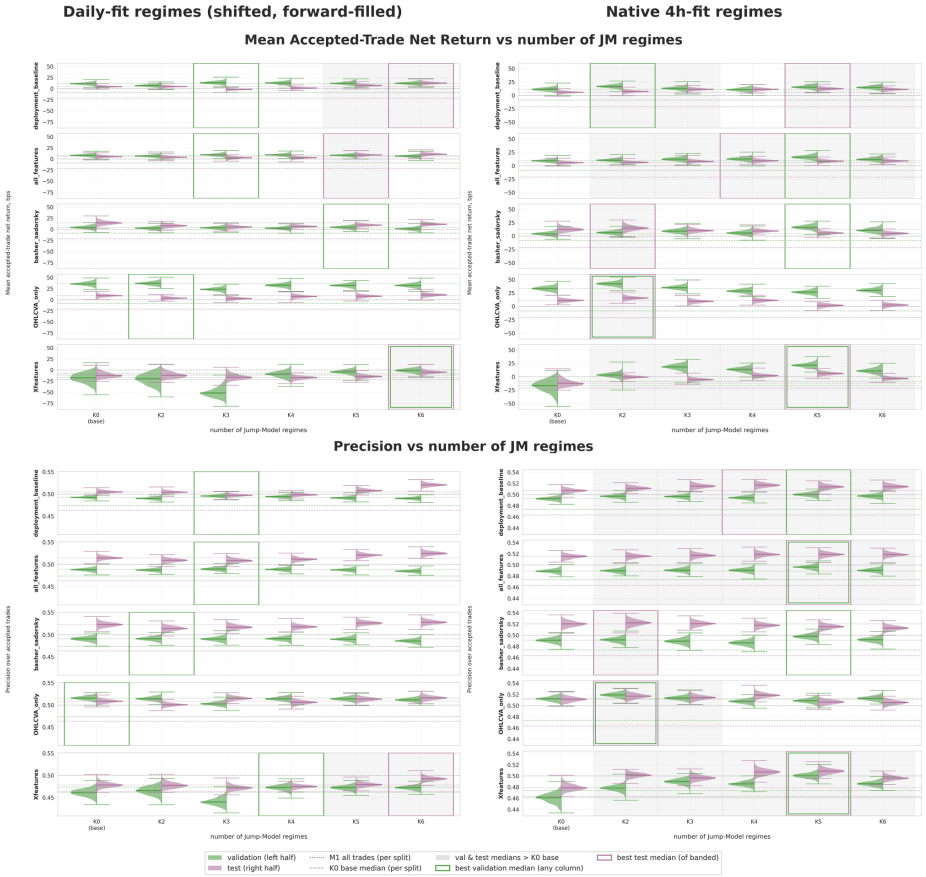


Fig. 18. Mean accepted-trade net return (top) and precision (bottom) over feature sets and regime features, daily-fit forward-filled regimes (left) and native 4h-fit regimes (right).

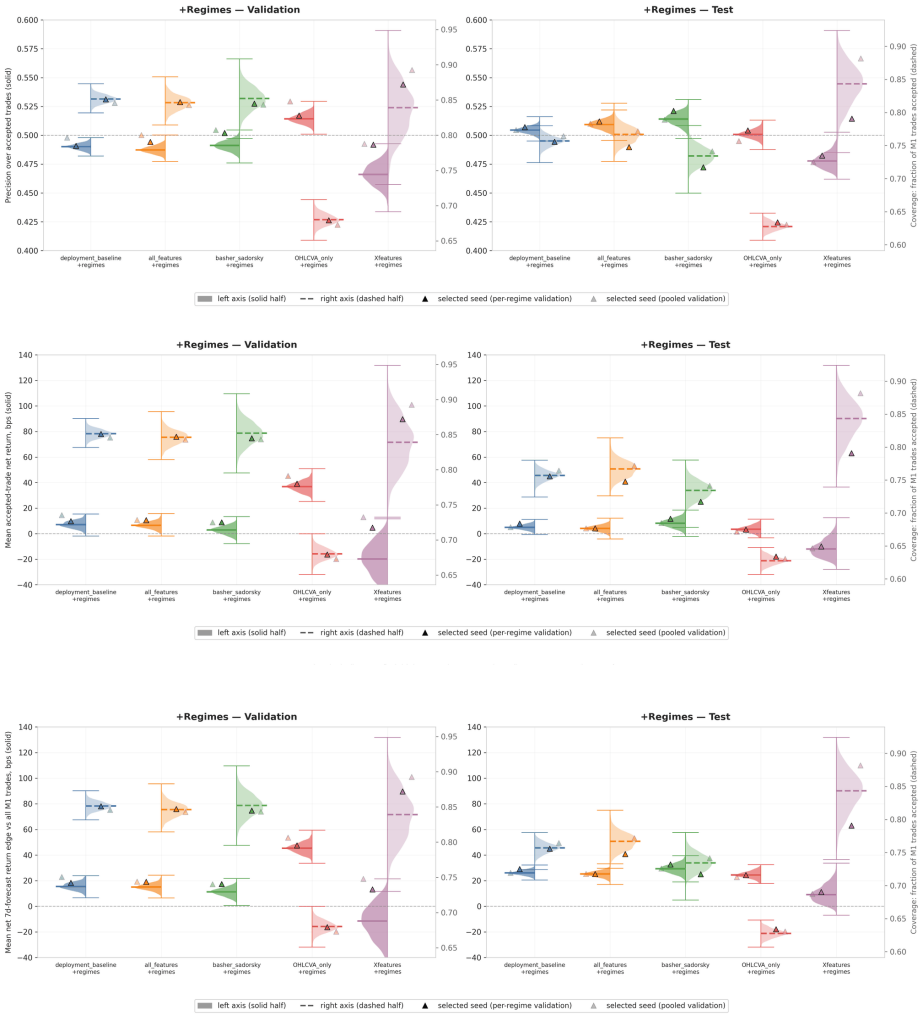


Fig. 19. Seed selection over the model clouds (K2 - Daily-fit forward-filled regimes).

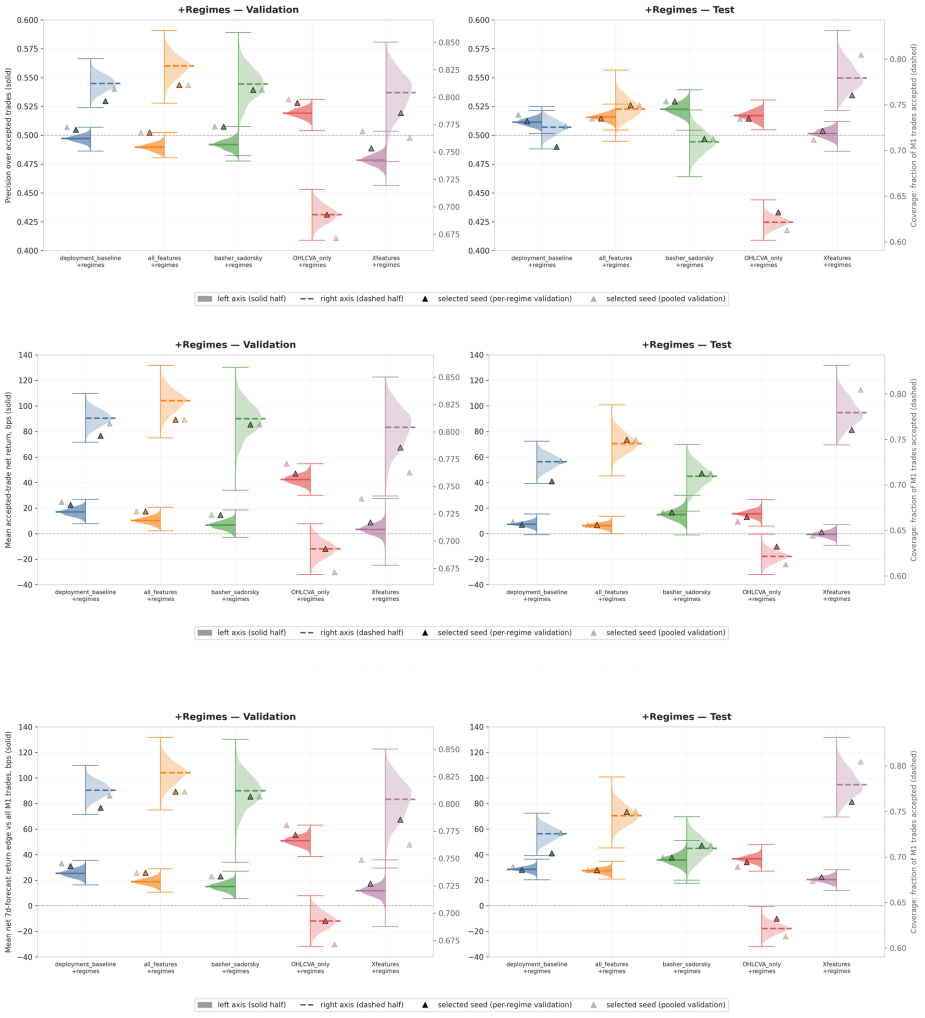


Fig. 20. Seed selection over the model clouds (K2 - Native 4h-fit regimes).

A.5 Results on the walk-forward cross-validation splits

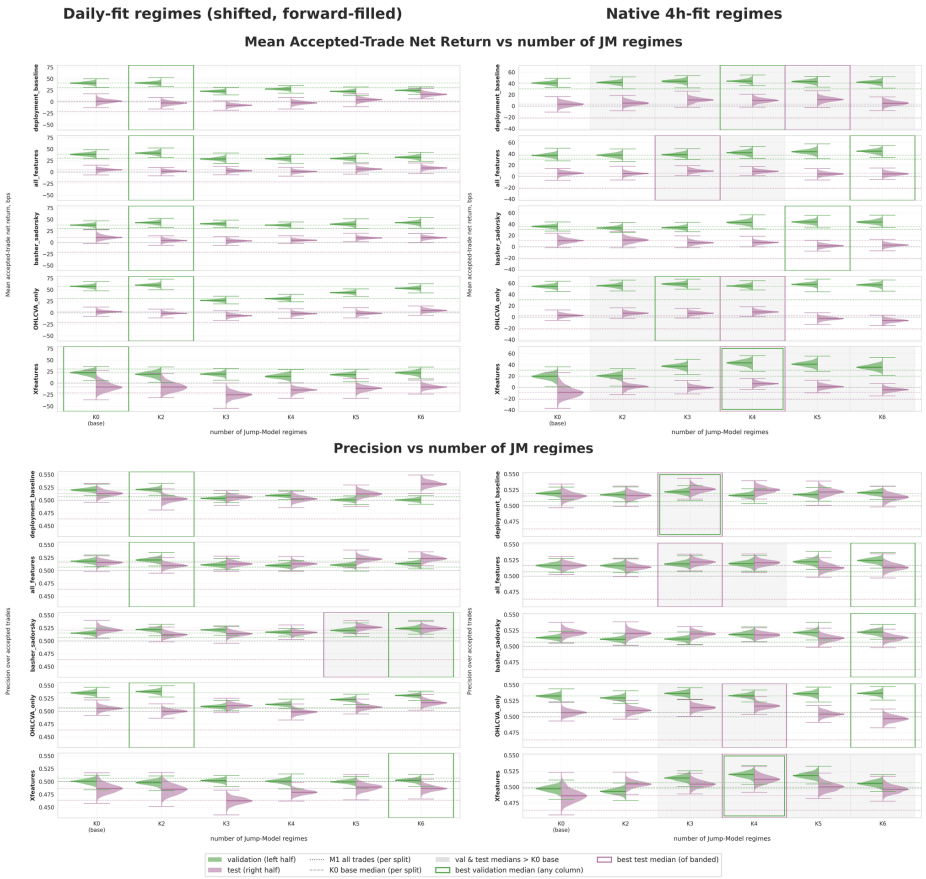


Fig. 21. Mean accepted-trade net return (top) and precision (bottom) over feature sets and regime features under walk-forward cross-validation, daily-fit forward-filled regimes (left) and native 4h-fit regimes (right).

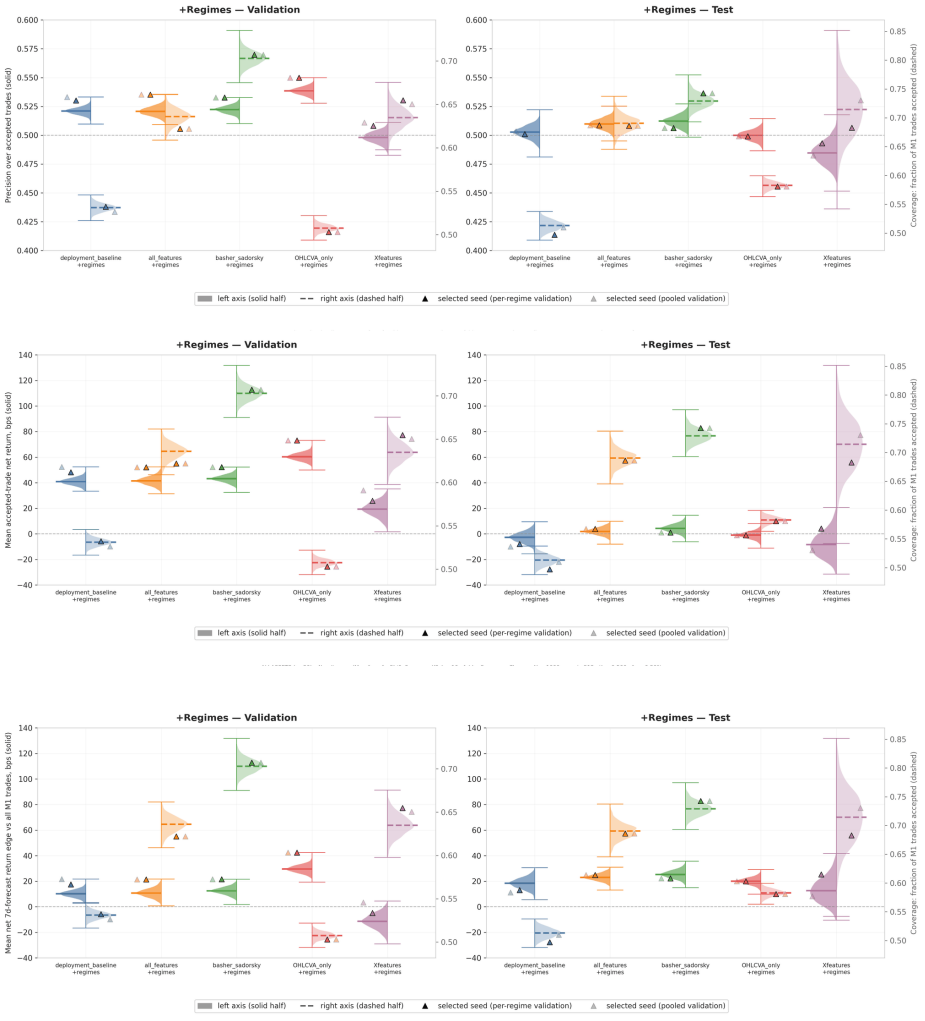


Fig. 22. Seed selection over the model clouds (K2 - Daily-fit forward-filled regimes).

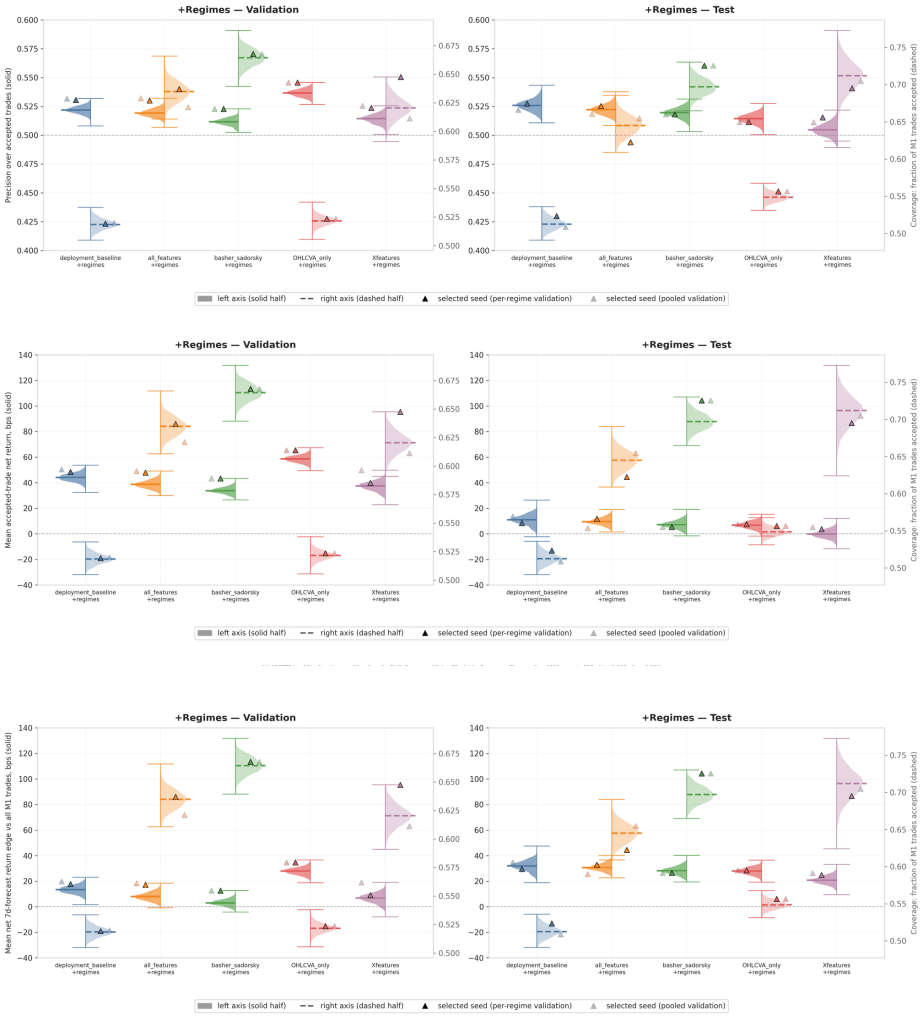


Fig. 23. Seed selection over the model clouds (K3 - Native 4h-fit regimes).

Glossary

active return The return of an asset or factor relative to a benchmark, calculated as its return minus the return of the benchmark. 17

Average Precision A performance metric that summarizes the precision–recall curve, measuring how well a model ranks positive observations ahead of negative ones. Higher values indicate better performance. 26

bear market Financial market period characterized by high risk and falling asset prices. During this time, investors expect values to drop further, and market sentiment is generally negative. 11

bull market Financial market period where prices rise consistently over time, typically by 20% or more. It is driven by investor optimism and a strong, growing economy. 11

causal z-scores Standardisation of x_t using the mean and standard deviation of a trailing window ending at t, preventing look-ahead bias and locally adapting to non-stationary scale changes. 4

CCi30 A market index designed to track the overall performance of a diversified group of major cryptocurrencies, providing a benchmark for the broader cryptocurrency market. 17

concept drift Machine learning phenomenon where the relationship between input features and target variables changes over time, rendering trained models less accurate.

$$X \to Y \text{ changes } (\text{regardless of } X)$$

. 6, 26

covariate shift Machine learning phenomenon where the distribution of input features $P(X)$ changes, while the conditional probability $P(Y|X)$ (labeling rule) remains stable.

$$X \text{ changes}, \; X \to Y \text{ remains}$$

. 6

excess return The return of an asset above or below a risk-free rate, calculated as the asset return minus the risk-free rate. 17

feedback delay issue True values (ground truth) are only available after the forecast horizon (how far into the future the prediction is made). Thereby, the model is training on outdated data and may suffer from distribution shifts (especially when the prediction window is large). 9

forecast horizon Number of time series points H that the model tries to predict. 4, 18

foundation model Large-scale model trained on massive, diverse datasets that serve as a flexible base for a wide variety of downstream tasks. 8

K-line data Also called candlestick data, it summarizes the price movement and trading activity of a financial asset over a fixed time interval. Each K-line contains the Open, High, Low, and Close prices, together with trading Volume and Amount (Turnover), forming OHLCVA data. 4

label shift Machine learning phenomenon where the distribution of target variables $P(Y)$ changes, while the distribution of features given a specific label $P(X|Y)$ remains stable (i.e. each class retains the same feature distribution).

$$Y \textit{ changes},\ Y \to X \textit{ remains}$$

. 6

miscalibrated model A predictive model whose estimated probabilities do not accurately reflect observed outcome frequencies. As a result, a fixed classification threshold may be poorly chosen and lead to misleading performance, even when the model ranks observations reasonably well. 26

MSCI USA Index A stock market index that tracks the performance of large-sized and mid-sized companies in the United States and is commonly used as a benchmark for the US equity market. 17

non-stationarity Time series or stochastic process whose statistical properties, such as mean, variance, or autocorrelation, change over time. 5

online learning Machine learning method in which the model updates itself continuously using new observations (as opposed to batch learning where the model is trained once). 7

range-bound market A range-bound market is a market that moves sideways between two set prices. The price goes neither up nor down. To profit, traders use Range-Bound Trading: buying at the bottom and selling at the top. 11

time series Sequence of data points ordered by time, typically representing measurements or observations taken at successive, consistent time intervals. 3, 4

turnover The total monetary value of trades over a given period. For a USDT trading pair, turnover is measured in USDT and represents the value of the cryptocurrency traded. 17

References

[1] Arjovsky, M., Bottou, L., Gulrajani, I., Lopez-Paz, D.: Invariant Risk Minimization. *arXiv preprint arXiv:1907.02893* (2019). DOI: 10.48550/arXiv.1907.02893.

[2] Aydınhan, A. O., Kolm, P. N., Mulvey, J. M., Shu, Y.: Identifying Patterns in Financial Markets: Extending the Statistical Jump Model for Regime Identification. *Annals of Operations Research* (2024). DOI: 10.1007/s10479-024-06035-z. URL: https://doi.org/10.1007/s10479-024-06035-z.

[3] Basher, S. A., Sadorsky, P.: Forecasting Bitcoin Price Direction with Random Forests: How Important Are Interest Rates, Inflation, and Market Volatility? *Machine Learning with Applications* **9** (2022), p. 100355. DOI: 10.1016/j.mlwa.2022.100355.

[4] Bemporad, A., Breschi, V., Piga, D., Boyd, S. P.: Fitting Jump Models. *Automatica* **96** (2018), pp. 11–21. DOI: 10.1016/j.automatica.2018.06.022.

[5] Chen, M., Shen, L., Fu, H., Li, Z., Sun, J., Liu, C.: Calibration of Time-Series Forecasting: Detecting and Adapting Context-Driven Distribution Shift. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2024. DOI: 10.1145/3637528.3671926.

[6] Cortese, F. P., Kolm, P. N., Lindström, E.: What Drives Cryptocurrency Returns? A Sparse Statistical Jump Model Approach. *Digital Finance* **5** (2023), pp. 483–518. DOI: 10.1007/s42521-023-00085-x.

[7] Cortese, F. P., Kolm, P. N., Lindström, E.: Generalized Information Criteria for High-Dimensional Sparse Statistical Jump Models. *AStA Advances in Statistical Analysis* **110** (2026), pp. 289–317. DOI: 10.1007/s10182-026-00554-9. URL: https://doi.org/10.1007/s10182-026-00554-9.

[8] de Prado, M.: Advances in Financial Machine Learning. Wiley, 2018. ISBN: 978-1-119-48210-9.

[9] Ding, Y., Kambouroudis, D., McMillan, D. G.: Forecasting Realised Volatility Using Regime-Switching Models. *International Review of Economics & Finance* **101** (2025), p. 104171. DOI: 10.1016/j.iref.2025.104171.

[10] Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., et al.: AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. In: *ICML Workshop on Automated Machine Learning*. 2020.

[11] Gibbs, I., Candès, E. J.: Conformal Inference for Online Prediction with Arbitrary Distribution Shifts. *Journal of Machine Learning Research* **25**.162 (2024), pp. 1–36.

[12] Gomes, H. M., Bifet, A., Read, J., Barddal, J. P., Enembreck, F., Pfahringer, B., et al.: Adaptive Random Forests for Evolving Data Stream Classification. *Machine Learning* **106**.9 (2017), pp. 1469–1495. DOI: 10.1007/s10994-017-5642-8.

[13] Gomes, H. M., Read, J., Bifet, A.: Streaming Random Patches for Evolving Data Stream Classification. In: *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2019, pp. 240–249.

[14] Gunasekara, N., Pfahringer, B., Gomes, H., Bifet, A.: Gradient Boosted Trees for Evolving Data Streams. *Machine Learning* **113** (2024), pp. 3325–3352. DOI: 10.1007/s10994-024-06517-y.

[15] Hollmann, N., Müller, S., Eggensperger, K., Hutter, F.: TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. In: *International Conference on Learning Representations*. 2023.

[16] Kulinski, S. M., Bagchi, S., Inouye, D. I.: Feature Shift Detection: Localizing Which Features Have Shifted via Conditional Distribution Tests. In: *Advances in Neural Information Processing Systems*. 2020.

[17] Nystrup, P., Kolm, P. N., Lindström, E.: Greedy Online Classification of Persistent Market States Using Realized Intraday Volatility Features. *The Journal of Financial Data Science* **2**.3 (2020), pp. 25–39. DOI: 10.3905/jfds.2020.2.3.025.

[18] Nystrup, P., Kolm, P. N., Lindström, E.: Feature Selection in Jump Models. *Expert Systems with Applications* **184** (2021), p. 115558. DOI: 10.1016/j.eswa.2021.115558. URL: https://doi.org/10.1016/j.eswa.2021.115558.

[19] Nystrup, P., Lindström, E., Madsen, H.: Learning Hidden Markov Models with Persistent States by Penalizing Jumps. *Expert Systems with Applications* **150** (2020), p. 113307. DOI: 10.1016/j.eswa.2020.113307.

[20] Oprea, S.-V., Bâra, A.: Regime-Aware Adaptive Forecasting Framework for Bitcoin Prices Using Probabilistic Generative Models. *Computational Economics* (2026). DOI: 10.1007/s10614-026-11338-3.

[21] Paye, B. S., Timmermann, A.: Instability of Return Prediction Models. *Journal of Empirical Finance* **13**.3 (2006), pp. 274–315. DOI: 10.1016/j.jempfin.2005.11.001.

[22] Qu, J., Holzmüller, D., Varoquaux, G., Le Morvan, M.: TabICL: A Tabular Foundation Model for In-Context Learning on Large Data. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 50817–50847.

[23] Shi, Y., Fu, Z., Chen, S., Zhao, B., Xu, W., Zhang, C., et al.: Kronos: A Foundation Model for the Language of Financial Markets. *Proceedings of the AAAI Conference on Artificial Intelligence* **40**.30 (2026), pp. 25366–25373. DOI: 10.1609/aaai.v40i30.39730. URL: https://doi.org/10.1609/aaai.v40i30.39730.

[24] Shu, Y.: Modeling Regime Changes in Financial Markets Using Statistical Jump Models: Methodology and Applications. PhD dissertation. Princeton University, 2025.

[25] Shu, Y., Mulvey, J. M.: Dynamic Factor Allocation Leveraging Regime-Switching Signals. *The Journal of Portfolio Management* **51**.3 (2025), pp. 50–72. DOI: 10.3905/jpm.2024.1.649.

[26] Shu, Y., Yu, C., Mulvey, J. M.: Downside Risk Reduction Using Regime-Switching Signals: A Statistical Jump Model Approach. *Journal of Asset Management* **25**.5 (2024), pp. 493–507. DOI: 10.1057/s41260-024-00376-x.

[27] Welch, I., Goyal, A.: A Comprehensive Look at the Empirical Performance of Equity Premium Prediction. *The Review of Financial Studies* **21**.4 (2008), pp. 1455–1508. DOI: 10.1093/rfs/hhm014.

[28] Wu, X., Teng, F., Li, X., Zhang, J., Duan, Q., Li, T.: Out-of-Distribution Generalization in Time Series: A Survey. *Information Fusion* **133** (2026), p. 104336. DOI: 10.1016/j.inffus.2026.104336.

[29] Zhao, L., Shen, Y.: Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*. ACM, 2025. DOI: 10.1145/3690624.3709210.